



*Asociación Profesional de Cuerpos Superiores
de Sistemas y Tecnologías de la Información
de las Administraciones Públicas.*

Temario para la preparación de la Oposición al Cuerpo Superior de
Sistemas y Tecnologías de la Información de la Administración del
Estado.

TEMAS ESPECÍFICOS – BLOQUE B

II. Tecnología básica

**Tema 63. Los sistemas de gestión de bases de datos SGBD.
El modelo de referencia de ANSI.**

AUTOR: Alfonso Sancho Miró.

REVISOR: Leonor Torres Moreno.

Creación: septiembre 2023

Actualización:

ÍNDICE

1	INTRODUCCIÓN.	4
2	CONCEPTO DE SISTEMA DE GESTIÓN DE BASES DE DATOS (SGBD).	6
1.1	DEFINICIÓN.	6
1.2	ACTORES.	6
1.2.1	ADMINISTRADOR DE BASES DE DATOS.	6
1.2.1.1	DBA DEL SISTEMA.	7
1.2.1.2	ARQUITECTO DE BASES DE DATOS.	7
1.2.1.3	ANALISTA DE BASE DE DATOS.	7
1.2.1.4	DBA DE APLICACIONES.	7
1.2.1.5	DBA ORIENTADOS A TAREAS.	8
1.2.1.6	ANALISTAS DE DESEMPEÑO.	8
1.2.1.7	ADMINISTRADOR DE ALMACÉN DE DATOS.	8
1.2.1.8	DBA EN LA NUBE.	8
1.2.2	DISEÑADORES DE BASES DE DATOS.	8
1.2.3	INGENIEROS DE SOFTWARE.	9
1.2.4	USUARIOS FINALES.	9
1.3	VENTAJAS EN EL USO DE LOS SGBD.	9
1.3.1	CONTROL DE REDUNDANCIA.	9
1.3.2	CONTROL DE ACCESO.	9
1.3.3	ALMACENAMIENTO PERSISTENTE.	9
1.3.4	MÚLTIPLES INTERFACES DE USUARIO.	9
1.3.5	REPRESENTACIÓN DE VÍNCULOS COMPLEJOS ENTRE LOS DATOS.	9
1.3.6	RESTRICCIONES DE INTEGRIDAD.	10
1.3.7	COPIAS DE SEGURIDAD Y RECUPERACIÓN.	10
1.4	RESUMEN DEL APARTADO.	10
2	EL MODELO DE REFERENCIA DE ANSI.	11
2.1	DEFINICIÓN.	11
2.2	INDEPENDENCIA DE DATOS.	12
2.3	RESUMEN DEL APARTADO.	13
3	ARQUITECTURA DE UN SGBD.	14
3.1	EVOLUCIÓN HISTÓRICA.	14
3.2	MODELOS DE BASES DE DATOS.	14
3.2.1	MODELO JERÁRQUICO.	14
3.2.2	MODELO EN RED (CODASYL).	15
3.2.3	MODELO RELACIONAL.	16
3.2.4	MODELO ORIENTADO A OBJETOS Y MODELO OBJETO-RELACIONAL.	17
3.2.5	BASES DE DATOS DOCUMENTALES Y BASES DE DATOS MULTIMEDIA.	18
3.2.6	MODELOS NOSQL.	19
3.2.6.1	MODELOS ORIENTADOS A DOCUMENTOS.	19
3.2.6.2	MODELOS ORIENTADOS A COLUMNAS.	19
3.2.6.3	MODELO CLAVE-VALOR.	20
3.2.6.4	MODELOS DE GRAFOS.	20
3.2.6.5	VENTAJAS E INCONVENIENTES DE NOSQL.	20
3.3	ARQUITECTURAS DE LOS SGBD.	20

3.3.1	ARQUITECTURA DEL SISTEMA.	20
3.3.1.1	SGBD CENTRALIZADO.	21
3.3.1.2	SGBD CLIENTE/SERVIDOR.	21
3.3.2	ARQUITECTURA DE DISTRIBUCIÓN DE DATOS.	21
3.3.3	ARQUITECTURA INTERNA: ELEMENTOS DEL SGBD.	22
3.3.3.1	LENGUAJES DEL SGBD.	22
3.3.3.2	EL DICCIONARIO DE DATOS.	23
3.3.3.3	INTERFACES DEL SGBD.	23
3.3.3.4	MÓDULOS COMPONENTES.	23
3.3.3.5	UTILIDADES.	24
3.3.4	ARQUITECTURA DE REFERENCIA.	24
3.4	RESUMEN DEL APARTADO.	24
4	GESTOR TRANSACCIONAL. CONTROL DE CONCURRENCIA. BLOQUEOS. RECUPERACIÓN DE ERRORES. INTEGRIDAD.	25
4.1	GESTOR TRANSACCIONAL.	25
4.2	CONTROL DE CONCURRENCIA. BLOQUEOS.	25
4.2.1	LECTURA SUCIA.	25
4.2.2	MODIFICACIÓN PERDIDA.	26
4.2.3	LECTURA NO REPETIBLE.	27
4.2.4	LECTURA FANTASMA.	27
4.2.5	BLOQUEOS.	28
4.2.5.1	NIVELES DE AISLAMIENTO.	28
4.3	RECUPERACIÓN DE ERRORES.	28
4.4	INTEGRIDAD.	29
4.5	RESUMEN DEL APARTADO.	29
5	PRINCIPALES SGBD UTILIZADOS EN LA ADMINISTRACIÓN.	31
5.1	ORACLE.	31
5.1.1	CONCEPTOS BÁSICOS.	31
5.1.2	ARQUITECTURA.	31
5.1.3	ADMINISTRACIÓN, GESTIÓN Y OPTIMIZACIÓN.	31
5.2	POSTGRESQL.	32
5.2.1	CONCEPTOS BÁSICOS.	32
5.2.2	ARQUITECTURA.	32
5.2.3	ADMINISTRACIÓN, GESTIÓN Y OPTIMIZACIÓN.	33
5.3	MYSQL / MARIADB.	33
5.3.1	CONCEPTOS BÁSICOS.	33
5.3.2	ARQUITECTURA.	33
5.3.3	ADMINISTRACIÓN, GESTIÓN Y OPTIMIZACIÓN.	33
6	TENDENCIAS ACTUALES.	34
6.1	PROCESAMIENTO HÍBRIDO TRANSACCIONAL-ANALÍTICO (HTAP, <i>HYBRID TRANSACTIONAL/ANALYTICAL PROCESSING</i>).	34
6.2	BASES DE DATOS MULTIMODELO.	34
6.3	SEGURIDAD INCORPORADA.	35
6.4	BASES DE DATOS INTELIGENTES Y AUTÓNOMAS.	35

6.5	CONCLUSIONES.	35
7	RESUMEN ESQUEMÁTICO.	36
8	GLOSARIO.	37
9	TEST.	38
9.1	PREGUNTAS DE TEST.	38
9.2	SOLUCIONES A LAS PREGUNTAS DE TEST.	41
10	BIBLIOGRAFÍA BÁSICA.	42
10.1	BIBLIOGRAFÍA BÁSICA.	42
10.2	BIBLIOGRAFÍA PARA AMPLIAR EL TEMA.	42

1 Introducción.

Las bases de datos y su tecnología es uno de los conceptos que más impacto ha tenido en el uso de las tecnologías de la información. Las bases de datos desempeñan un papel fundamental en casi todas las áreas donde se utilizan tecnologías de la información, incluyendo negocios, ingeniería, medicina, leyes, educación y, por supuesto, la administración pública.

Definimos un Sistema Gestor de Bases de Datos o SGBD, también llamado DBMS (*Data Base Management System*) como una colección de datos relacionados entre sí, estructurados y organizados, y un conjunto de programas que acceden y gestionan esos datos. La colección de esos datos se denomina Base de Datos o BD.

Antes de aparecer los SGBD (década de los setenta del siglo XX), la información se trataba y se gestionaba utilizando los típicos sistemas de gestión de archivos que iban soportados sobre un sistema operativo. Éstos consistían en un conjunto de programas que definían y trabajaban sus propios datos. Los datos se almacenan en archivos y los programas manejan esos archivos para obtener la información. Si la estructura de los datos de los archivos cambia, todos los programas que los manejan se deben modificar; por ejemplo, un programa trabaja con un archivo de datos de alumnos, con una estructura o registro ya definido; si se incorporan elementos o campos a la estructura del archivo, los programas que utilizan ese archivo se tienen que modificar para tratar esos nuevos elementos. En estos sistemas de gestión de archivos, la definición de los datos se encuentra codificada dentro de los programas de aplicación en lugar de almacenarse de forma independiente, y además el control del acceso y la manipulación de los datos viene impuesto por los programas de aplicación.

Esto supone un gran inconveniente a la hora de tratar grandes volúmenes de información. Surge así la idea de separar los datos contenidos en los archivos de los programas que los manipulan, es decir, que se pueda modificar la estructura de los datos de los archivos sin que por ello se tengan que modificar los programas con los que trabajan. Se trata de estructurar y organizar los datos de forma que se pueda acceder a ellos con independencia de los programas que los gestionan.

Inconvenientes de un sistema de gestión de archivos:

- Redundancia e inconsistencia de los datos, se produce porque los archivos son creados por distintos programas y van cambiando a lo largo del tiempo, es decir, pueden tener distintos formatos y los datos pueden estar duplicados en varios sitios. Por ejemplo, el teléfono de un alumno puede aparecer en más de un archivo. La redundancia aumenta los costes de almacenamiento y acceso, y trae consigo la inconsistencia de los datos: las copias de los mismos datos no coinciden por aparecer en varios archivos.
- Dependencia de los datos física-lógica, o lo que es lo mismo, la estructura física de los datos (definición de archivos y registros) se encuentra codificada en los programas de aplicación. Cualquier cambio en esa estructura implica al programador identificar, modificar y probar todos los programas que manipulan esos archivos.
- Dificultad para tener acceso a los datos, proliferación de programas, es decir, cada vez que se necesite una consulta que no fue prevista en el inicio implica la necesidad de codificar el programa de aplicación necesario. Lo que se trata de probar es que los entornos convencionales de procesamiento de archivos no permiten recuperar los datos necesarios de una forma conveniente y eficiente.
- Separación y aislamiento de los datos, es decir, al estar repartidos en varios archivos, y tener diferentes formatos, es difícil escribir nuevos programas que aseguren
- la manipulación de los datos correctos. Antes se deberían sincronizar todos los archivos para que los datos coincidiesen.
- Dificultad para el acceso concurrente, pues en un sistema de gestión de archivos es complicado que los usuarios actualicen los datos simultáneamente. Las actualizaciones concurrentes pueden dar por resultado datos inconsistentes, ya que se puede acceder a los datos por medio de diversos programas de aplicación.
- Dependencia de la estructura del archivo con el lenguaje de programación, pues la estructura se define dentro de los programas. Esto implica que los formatos de los archivos sean incompatibles. La incompatibilidad entre archivos generados por distintos lenguajes hace que los datos sean difíciles de procesar.
- Problemas en la seguridad de los datos. Resulta difícil implantar restricciones de seguridad pues las aplicaciones se van añadiendo al sistema según se van necesitando.
- Problemas de integridad de datos, es decir, los valores almacenados en los archivos deben cumplir con restricciones de consistencia. Por ejemplo, no se puede insertar una nota de un

CONTENIDO

alumno en una asignatura si previamente esa asignatura no está creada. Otro ejemplo, las unidades en almacén de un producto determinado no deben ser inferiores a una cantidad. Esto implica añadir gran número de líneas de código en los programas. El problema se complica cuando existen restricciones que implican varios datos en distintos archivos.

Todos estos inconvenientes hacen posible el fomento y desarrollo de SGBD. El objetivo primordial de un gestor es proporcionar eficiencia y seguridad a la hora de extraer o almacenar información en las BD. Los SGBD están diseñados para gestionar grandes bloques de información, que implica tanto la definición de estructuras para el almacenamiento como de mecanismos para la gestión de la información.

Una BD es un gran almacén de datos que se define una sola vez; los datos pueden ser accedidos de forma simultánea por varios usuarios; están relacionados y existe un número mínimo de duplicidad; además en las bases de datos se almacenarán las descripciones de esos datos, lo que se llama metadatos en el diccionario de datos, que se verá más adelante.

2 Concepto de Sistema de Gestión de Bases de Datos (SGBD).

1.1 Definición.

Un sistema de gestión de bases de datos (SGBD, en inglés *database management system* o DBMS) es una colección de programas que permiten a los usuarios crear y mantener una base de datos. El SGBD es por tanto un sistema software de propósito general que facilita los procesos de definición, construcción y manipulación de bases de datos para distintas aplicaciones.

- La definición de una base de datos consiste en especificar los tipos de datos, las estructuras y restricciones para los datos que se van a almacenar en dicha base.
- La construcción de la base de datos es el proceso de almacenar los datos concretos sobre algún medio de almacenamiento controlado por el SGBD.
- La manipulación de la base de datos incluye funciones tales como consultarla para recuperar datos específicos, actualizar los datos y generar informes.

Hay varias características que distinguen el enfoque de los SGBD del enfoque basado en programación con ficheros. En el procesamiento de ficheros, cada usuario define e implementa los ficheros para una aplicación específica como parte de la programación de la aplicación. Sin embargo, en el enfoque de un SGBD, se mantiene un único almacén de datos que se define una sola vez y posteriormente es accedido por varios usuarios. Las principales características del enfoque de los SGBD son las siguientes:

- Los sistemas de bases de datos tienen naturaleza autodescriptiva. El SGBD no contiene únicamente la base de datos, sino que también contiene una definición completa de su estructura y restricciones, la cual se almacena en el catálogo del sistema.
- Existe separación entre los programas y los datos.
- Soporte de múltiples vistas de los datos. Una base de datos suele tener muchos usuarios, por lo que los SGBD ofrecen mecanismos para definir múltiples vistas. Una vista puede ser un subconjunto de la base de datos o puede contener datos virtuales derivados de la base de datos.
- Compartición de datos y procesamiento de transacciones multiusuario. Todo SGBD multiusuario debe permitir a varios usuarios tener acceso simultáneo a la base de datos, asegurando el control de la concurrencia.

1.2 Actores.

1.2.1 Administrador de bases de datos.

El administrador de bases de datos o DBA (de sus siglas en inglés, *database administrator*) se encarga de supervisar y gestionar la base de datos, el SGBD y el software relacionado con él. El DBA autoriza el acceso a la base de datos; coordina y vigila su utilización; y adquiere los recursos de software y hardware que sean necesarios. El DBA es la persona responsable cuando surgen problemas como violaciones de la seguridad o una respuesta lenta del sistema.

Sus tareas incluyen las siguientes:

- Implementar, dar soporte y gestionar bases de datos corporativas.
- Crear y configurar bases de datos.
- Ser responsables de la integridad y la disponibilidad de los datos.
- Diseñar, desplegar y monitorizar servidores de bases de datos.
- Diseñar la distribución de los datos y las soluciones de almacenamiento.
- Garantizar la seguridad de las bases de datos, realizar copias de seguridad y llevar a cabo la recuperación de desastres.
- Planificar e implementar el aprovisionamiento de los datos y aplicaciones.
- Diseñar planes de contingencia.
- Diseñar y crear las bases de datos corporativas de soluciones avanzadas.
- Analizar y reportar datos corporativos que ayuden a la toma de decisiones en la inteligencia de negocios.
- Producir diagramas de entidades relacionales y diagramas de flujos de datos, normalización esquemática, localización lógica y física de bases de datos.

Dentro de organizaciones grandes, las responsabilidades de **DBA** generalmente se dividen en tipos de trabajo separados.

Los tipos principales de administradores de bases de datos son:

- De sistemas
- Arquitecto de bases de datos
- Analista de bases de datos
- De aplicaciones
- Orientado a tareas
- Analista de rendimiento
- Administrador de almacenamiento de datos
- En la nube.

La anterior no es necesariamente una lista exhaustiva de todos los tipos de DBA. También es importante tener en cuenta que no todas las organizaciones tienen cada uno de estos tipos de DBA separados.

1.2.1.1 DBA del sistema.

Esta función se centra en cuestiones técnicas, más que comerciales. El administrador de bases de datos del sistema conoce los detalles técnicos secretos de cómo se instala, configura y modifica la base de datos. Las tareas típicas se centran en la instalación física y el rendimiento del software SGBD y pueden incluir:

- Instalar nuevas versiones y aplicar correcciones.
- Configuración y ajuste de los parámetros del sistema.
- Ajustar el sistema operativo, la red y los procesadores de transacciones para que funcionen con el SGBD.
- Asegurar que el almacenamiento y la memoria adecuados estén disponibles para el SGBD.
- Los administradores de bases de datos del sistema rara vez participan en la implementación real de la base de datos y la aplicación. Pueden involucrarse en el ajuste de la aplicación cuando sea necesario modificar parámetros del sistema operativo o parámetros complejos del SGBD.

1.2.1.2 Arquitecto de bases de datos.

Esta persona trabaja principalmente en el diseño e implementación de nuevas bases de datos. El arquitecto de bases de datos diseña nuevas bases de datos y estructuras para aplicaciones nuevas o existentes y rara vez participa en el mantenimiento y ajuste de bases de datos y aplicaciones establecidas. Las tareas típicas realizadas por el arquitecto de la base de datos incluyen:

- Modelado de datos lógicos.
- Traducir modelos de datos lógicos en un diseño de base de datos física.
- Analizar los requisitos de acceso a los datos para garantizar un diseño óptimo de la base de datos y un acceso SQL eficiente.
- Creación de estrategias de respaldo y recuperación para nuevas bases de datos.

1.2.1.3 Analista de base de datos.

A veces, los DBA junior se denominan analistas de bases de datos. Un analista de bases de datos puede desempeñar una función similar a la del arquitecto de bases de datos. Un analista de bases de datos también puede ser simplemente otro término que utilizan las empresas en lugar de administrador de bases de datos.

1.2.1.4 DBA de Aplicaciones.

Esta función se centra en el diseño de la base de datos y el soporte y la administración continuos de la base de datos para una aplicación o subconjunto de aplicaciones específico. Es más probable que sean expertos en escribir y depurar SQL complejo y comprender las mejores formas de incorporar solicitudes de bases de datos en programas de aplicación.

No todas las organizaciones cuentan con administradores de bases de datos de aplicaciones. Sin embargo, cuando existen DBA de aplicaciones, todavía se requieren DBA de propósito general para

Comentado [MLTM1]: Habría que darle una vuelta para no repetir tanto DBA y que queden. El párrafo está repetido

Comentado [AS2R1]: Lo he reorganizado para que no sea repetitivo. He borrado el párrafo duplicado.

admitir el entorno y la infraestructura de la base de datos en general. Cuando no existen DBA de aplicaciones, la organización suele tener DBA de propósito general que admiten aplicaciones específicas y, al mismo tiempo, mantienen el entorno de la base de datos de la organización.

1.2.1.5 DBA orientados a tareas.

Estos son DBA especializados que se enfocan en una tarea administrativa específica. Los DBA orientados a tareas son poco comunes fuera de las grandes organizaciones de TI. Un ejemplo de un DBA orientado a tareas es un DBA de respaldo y recuperación que garantiza que las bases de datos de la organización sean recuperables, incluida la creación de planes de respaldo; construir y probar scripts de respaldo; probar scripts de recuperación; y realizar tareas de recuperación cuando sea necesario. El DBA de respaldo y recuperación también participa en la creación y prueba de planes de contingencia ante desastres para las bases de datos de la organización.

1.2.1.6 Analistas de desempeño.

Este es otro tipo específico de DBA orientado a tareas. El analista de rendimiento es el más común de los DBA orientados a tareas. Este rol se enfoca completamente en monitorizar y mejorar el desempeño de las aplicaciones que acceden a las bases de datos.

Un analista de rendimiento es un experto en codificación SQL para el rendimiento, además de tener conocimientos sobre el diseño y la creación de bases de datos de alto rendimiento. Un analista de rendimiento debe tener un conocimiento profundo del SGBD y ser capaz de interactuar con otros DBA para implementar los cambios apropiados cuando sea necesario.

El analista de rendimiento debe poder hablar con los desarrolladores de aplicaciones en su idioma para facilitar los cambios de programa adecuados para el rendimiento.

1.2.1.7 Administrador de almacén de datos.

Este es un DBA totalmente capaz, pero con conocimientos y habilidades específicos para monitorizar y respaldar el entorno de almacenamiento de datos. Los administradores del almacén de datos deben comprender las diferencias entre una base de datos que admite OLTP y un almacén de datos. La administración del almacén de datos requiere experiencia con lo siguiente:

- Inteligencia empresarial (BI) y herramientas de consulta;
- Diseño de bases de datos especializadas para almacenamiento de datos;
- Conocimiento de tecnologías de almacenamiento de datos, como OLAP y esquema en estrella;
- y
- Habilidades ETL.

Los almacenes de datos se ven con más detalle en otro tema del temario.

1.2.1.8 DBA en la nube.

Este rol se ha vuelto más popular a medida que las organizaciones migran cargas de trabajo a la nube. Un DBA en la nube realiza muchas de las mismas tareas que un DBA de propósito general, pero para implementaciones de bases de datos en la nube como AWS y Microsoft Azure.

El administrador de bases de datos en la nube debe comprender los servicios que ofrece el proveedor de la nube, como respaldo y seguridad, para implementar bases de datos en la nube. Los administradores de bases de datos en la nube deben conocer la latencia, la tolerancia a fallos y la administración de costes.

1.2.2 Diseñadores de bases de datos.

Los diseñadores de bases de datos se encargan de identificar los datos que se almacenarán en la base de datos, así como de elegir las estructuras adecuadas para presentar y almacenar dichos datos. Estas tareas suelen realizarse antes de que se implemente la base de datos y se carguen los datos.

Tienen la responsabilidad de comunicarse con todos los futuros usuarios para comprender sus necesidades y presentar un diseño que cumpla con sus requisitos.

1.2.3 Ingenieros de software.

Los ingenieros de software determinan los requisitos de los usuarios finales y desarrollan especificaciones para transacciones programadas que satisfagan dichos requisitos. Posteriormente, los programadores implementan esas especificaciones en forma de programas y luego los prueban, depuran, documentan y mantienen. Para realizar dichas tareas, los ingenieros de software deben conocer a la perfección toda la gama de capacidades del SGBD.

1.2.4 Usuarios finales.

Los usuarios finales aquellos cuyo trabajo requiere acceder a la base de datos para consultarla, actualizarla y generar informes. Estos usuarios, por lo general, no accederán directamente a la base de datos, sino que lo harán a través de aplicaciones, programas y/o interfaces de consulta.

1.3 Ventajas en el uso de los SGBD.

En este apartado se analizan algunas de las principales ventajas de utilizar un SGBD.

1.3.1 Control de redundancia.

La redundancia, esto es, el almacenamiento de los mismos datos varias veces provoca varios problemas: duplicar el trabajo; desperdiciar espacio de almacenamiento; y posibles inconsistencias. En el enfoque tradicional de procesamiento de ficheros es muy frecuente que se den situaciones de redundancia. Sin embargo, las bases de datos almacenan cada dato lógico en un único lugar, evitando la redundancia, lo que evita la inconsistencia y ahorra espacio de almacenamiento.

No obstante, en algunos casos resulta conveniente aplicar una redundancia controlada para mejorar el rendimiento de la base de datos; por ejemplo, para mejorar el tiempo de respuesta de las consultas. El SGBD permite que esa redundancia esté controlada en todo momento de manera que no se produzcan inconsistencias.

Algunos modelos de bases de datos, como el paradigma NoSQL, fomentan la redundancia controlada para aumentar el rendimiento de las consultas.

1.3.2 Control de acceso.

Los SGBD cuentan con un subsistema de seguridad y autorización que permite controlar el tipo de operaciones de acceso a la base de datos. Este sistema se basa en cuentas de usuario a las que se le conceden permisos de recuperación y/o actualización de datos.

1.3.3 Almacenamiento persistente.

Las bases de datos proporcionan almacenamiento persistente a los objetos y estructuras de datos de los programas y los sistemas de información, de manera que los datos permanecen almacenados, aunque el sistema de información no se esté ejecutando.

1.3.4 Múltiples interfaces de usuario.

Los SGBD suelen ofrecer diferentes interfaces para los distintos tipos de usuario, en función de su nivel de conocimientos técnicos. Entre estas, cabe destacar los lenguajes de consulta, las interfaces de programación (API), los formularios e incluso el acceso a la base de datos a través de la web.

1.3.5 Representación de vínculos complejos entre los datos.

Los SGBD tienen la capacidad de representar diversas relaciones complejas entre los datos y también obtener y actualizar con rapidez y eficiencia los datos que están mutuamente relacionados.

1.3.6 Restricciones de integridad.

La mayoría de las bases de datos tienen restricciones de integridad que deben cumplir los datos. Los SGBD ofrecen recursos para definir tales restricciones y garantizar que se cumplen.

1.3.7 Copias de seguridad y recuperación.

Los SGBD cuentan con recursos para la recuperación de fallos de hardware o de software. El subsistema de copias de seguridad y recuperación es el responsable de llevar a cabo dicha recuperación.

1.4 Resumen del apartado.

Una base de datos es un conjunto de datos -entendidos como hechos registrados- relacionados entre sí. Un SGBD es un paquete de software general para implementar y mantener una base de datos. Las características que aporta el enfoque de los SGBD son las siguientes:

- Existencia de un catálogo o diccionario de datos.
- Independencia entre programas y datos.
- Abstracción de los datos.
- Soporte de múltiples vistas de usuario.
- Compartición de datos entre múltiples transacciones.

Los principales tipos de usuario de una base de datos son los administradores (DBA); los diseñadores; los ingenieros de software; y los usuarios finales.

Finalmente, las ventajas que ofrece el uso de los SGBD son las siguientes:

- Control de la redundancia.
- Restricción de accesos no autorizados.
- Almacenamiento persistente.
- Múltiples interfaces de usuario.
- Representación de vínculos complejos entre los datos.
- Restricciones de integridad.
- Copias de seguridad y recuperación.

2 El modelo de referencia de ANSI.

2.1 Definición.

El modelo de ANSI o ANSI/SPARC (*American National Standards Institute / Standards Planning And Requirements Committee*) es un estándar de diseño abstracto para un sistema de gestión de bases de datos propuesto por primera vez en 1975. La mayoría de los SGBD comerciales modernos se basan en este sistema. Sin embargo, nunca se ha publicado como estándar de iure, sino que se trata de un estándar de facto ampliamente adoptado por la industria.

A este modelo se le conoce también como arquitectura de tres niveles o esquemas. Su objetivo es separar las aplicaciones del usuario y la base de datos físicas. Los niveles definidos son los siguientes:

1. El nivel interno cuenta con un único **esquema** que describe la estructura física de almacenamiento de la base de datos. El esquema interno emplea un modelo de datos físico y describe todos los detalles para su almacenamiento, así como los caminos de acceso para la base de datos.
2. El nivel conceptual también tiene un solo esquema, que describe la estructura de datos de la base de datos completa para una comunidad de usuarios. El esquema conceptual oculta los detalles de las estructuras físicas de almacenamiento y se concentra en describir entidades, tipos de datos, restricciones, etc.
3. El nivel externo o de usuario incluye varios esquemas externos o vistas de usuario. Cada esquema externo describe la parte de la base de datos que interesa a un grupo de usuarios **determinado**. Las vistas son útiles por varias razones:
 - a. Proporcionan un poderoso mecanismo de seguridad, ocultando partes de la base de datos a ciertos usuarios.
 - b. Permiten que los usuarios accedan a los datos en el formato que ellos desean.
 - c. Las vistas proporcionan independencia de datos a nivel lógico, que también se da cuando se reorganiza el nivel conceptual.
 - d. Los cambios se reflejan automáticamente en todas las vistas afectadas.

La mayoría de los SGBD no separan los tres niveles completamente, pero en algunos de ellos se soporta, en cierta medida, la arquitectura de tres niveles. Algunos SGBD incluyen detalles del nivel físico en el nivel conceptual. En casi todos los SGBD que manejan vistas de usuario, los esquemas externos se definen en el mismo modelo de datos que describe la información a nivel conceptual. Con algunos SGBD es posible utilizar distintos modelos de datos en los niveles conceptual y externo.

Cabe destacar que los tres niveles no son más que descripciones de los datos; los únicos datos que existen realmente están en el nivel físico. En un SGBD basado en esta arquitectura, se debe transformar una solicitud de usuario expresada en términos de su esquema externo en una solicitud expresada en términos del esquema conceptual; y luego en una solicitud del esquema interno que se procesará sobre la base de datos almacenada. El proceso de transformar solicitudes y resultados de un nivel a otro se denomina correspondencia o transformación (*mapping*).

Comentado [MLTM3]: Evitar repetir interno y conceptual tan seguido. Te lo he puesto en amarillo

Comentado [AS4R3]: Ok, sonaba muy repetitivo. Lo he modificado.

Comentado [MLTM5]: Hablar de la independencia entre las vistas o evitar acceder a datos no permitidos.

Comentado [AS6R5]: He añadido unos puntos al respecto.

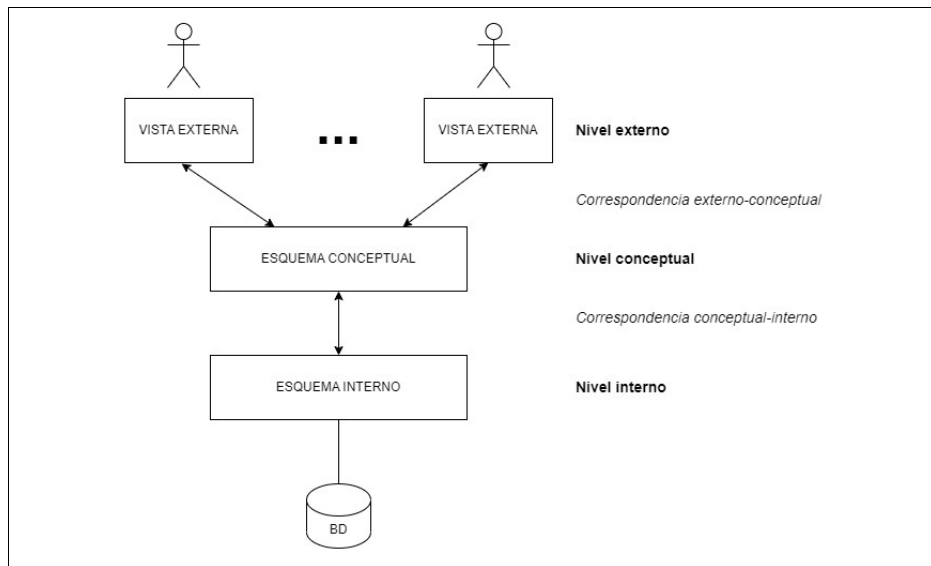


Figura 1: Arquitectura ANSI de tres niveles

2.2 Independencia de datos.

La independencia de datos es la capacidad para modificar el esquema en un nivel del sistema de base de datos sin tener que modificar el esquema del nivel inmediatamente superior. Existen dos tipos de independencia de datos:

1. La independencia lógica de los datos es la capacidad de modificar el esquema conceptual sin tener que alterar los esquemas externos ni los programas de aplicación. Podemos modificar el esquema conceptual para ampliar la base de datos o para reducirla. En el segundo caso, la modificación no deberá afectar a los esquemas externos que sólo se refieran a los datos restantes. Después de una reorganización lógica del esquema conceptual, los programas de aplicación que hagan referencia a los elementos del esquema externo deberán funcionar igual que antes. Además, las restricciones podrán modificarse en el esquema conceptual sin afectar a los esquemas externos ni a los programas de aplicación.
2. La independencia física de los datos es la capacidad de modificar el **esquema interno** sin alterar el esquema conceptual o los externos. Tal vez sea preciso modificar el esquema interno por la necesidad de reorganizar varios ficheros físicos con el fin de mejorar el rendimiento. Si la base de datos aún contiene los mismos datos, no será necesario modificar el esquema conceptual.

En todo SGBD de múltiples niveles es preciso ampliar el catálogo de modo que incluya información sobre cómo establecer la correspondencia entre las solicitudes y los datos entre los diversos niveles. El SGBD utiliza software adicional para realizar estas correspondencias, haciendo referencia a la información que se encuentra en el catálogo. La independencia de datos se logra porque, al modificarse el esquema en algún nivel, el esquema del nivel inmediatamente superior permanece sin cambios; sólo se modifica la correspondencia entre los dos niveles. Por tanto, no es preciso modificar los programas de aplicación que hacen referencia al esquema de nivel superior.

La arquitectura de tres niveles puede facilitar la consecución de la independencia de datos. Sin embargo, los dos niveles de correspondencia implican un gasto extra, lo cual reduce la eficiencia del SGBD. Es por ello por lo que son pocos SGBD los que realmente han implementado la arquitectura ANSI completa.

Comentado [MLTM7]: No será interno??

Comentado [AS8R7]: Ciertó, es el interno. Lo corrijo.

2.3 Resumen del apartado.

El modelo de ANSI o ANSI/SPARC es un estándar de diseño abstracto para SGBD, conocido también como arquitectura de tres niveles o esquemas. Su objetivo es separar las aplicaciones del usuario y la base de datos físicas. Define tres niveles: externo, conceptual e interno. Entre los niveles existen también correspondencias, que son mecanismos para transformar las solicitudes y resultados. Este modelo ayuda a alcanzar la independencia de datos, tanto lógica como física.

3 Arquitectura de un SGBD.

3.1 Evolución histórica.

El origen de las bases de datos se remonta a la antigüedad, cuando ya existían registros de hechos y situaciones, de forma escrita y gráfica. No obstante, a nivel informático, la primera vez de la que se tiene constancia del uso del término base de datos es en 1963 en California. En esta década aparece el SGBD *Information Management System* (IMS) de IBM, un gestor de bases de datos jerárquicas y un gestor transaccional con alta capacidad de proceso.

En la década de los 60 también comenzaron los trabajos de CODASYL (el acrónimo de *Conference on Data Systems Languages*), un consorcio de industrias informáticas formado que pretendían regular el desarrollo de un lenguaje de programación estándar que pudiera ser utilizado en multitud de ordenadores. Dichos trabajos tuvieron como resultado la creación del lenguaje COBOL. En 1969 se publicaron las primeras especificaciones para el modelo de base de datos en red, el cual acabó por ser conocido como Modelo CODASYL. Propiamente estas especificaciones definían varios lenguajes por separado: un lenguaje de descripción de datos (DDL, *Data Definition Language*) para definir el esquema de la base de datos; otro DDL para crear uno o más subesquemas para definir vistas de la base de datos en aplicaciones; y un lenguaje de manipulación de datos (DML, *Data Manipulation Language*) que definía palabras clave para incluir en el código COBOL las llamadas y actualizaciones de la base de datos.

En la década de los 70 se producen grandes avances. Codd define el modelo de datos relacional -el cual es objeto de otro tema- y, posteriormente, Chen crea el modelo entidad-relación. La Universidad de California creó el SGBD relacional *Ingress*. IBM crea el lenguaje de consulta SEQUEL (*Structured English Query Language*), para bases de datos relacionales. En 1979, *Oracle Corporation* introdujo por primera vez el SEQUEL en un SGBD comercial. Aparece también el SGBD *dBase*, para microordenadores.

En la década de los 80, ANSI estandariza el lenguaje SEQUEL, pasando a llamarse SQL (*Structured Query Language*). Microsoft lanza SQL Server. Algunos SGBD empiezan a incorporar soporte nativo a datos espaciales, como Oracle (con la extensión *Oracle Spatial*).

En los 90 aparecen los SGBD orientados a objetos. Microsoft lanza Access, muy utilizada a nivel doméstico y en organizaciones pequeñas y medianas. Comienzan dos proyectos de código abierto ampliamente utilizados en la actualidad: MySQL y PostgreSQL.

Ya en el siglo XXI, aparecen los primeros SGBD basados en el modelo NoSQL: *Cassandra*, *Redis*, *MongoDB*. Se extendió el uso de bases de datos cliente-servidor y bases de datos distribuidas.

Actualmente, algunos de los SGBD más conocidos y utilizados son *MySQL*, *MariaDB* y *PostgreSQL*, de código abierto; y *Oracle*, *Microsoft SQL Server* y *IBM DB2*, de tipo privativo. Todos ellos son de tipo relacional. En cuanto a sistemas no relacionales, los más utilizados son *MongoDB*, *Redis* y *Cassandra*, los tres de código abierto.

Los SGBD bases de datos continúan evolucionando hoy en día. Las principales tendencias son la integración de SGBD con sistemas de Big Data, los SGBD con soporte a múltiples modelos y los SGBD en la nube.

3.2 Modelos de bases de datos.

Un modelo de base de datos es la estructura lógica que adopta la base de base datos, incluyendo las relaciones y limitaciones que determinan cómo se almacenan y organizan y cómo se accede a los datos. El modelo de base de datos proporciona cierto nivel de abstracción de los datos, al ocultar detalles de almacenamiento que la mayoría de los usuarios no necesitan conocer.

A lo largo de la historia se han propuesto muchos modelos de bases de datos. En los siguientes apartados se describen los más representativos.

3.2.1 Modelo jerárquico.

No existe ningún documento original que describa el modelo jerárquico, como ocurren con los modelos relacional y en red. Los principios que sustentan el modelo jerárquico proceden del SGBD *Information*

Management System (IMS) de IBM, lanzado en la década de los 60 del siglo XX. Otro famoso SGBD jerárquico es el *System 2000* de MRI, posteriormente comercializado por *SAS Institute*.

Este modelo utiliza dos conceptos principales: registros y relaciones padre-hijo. Un registro es una colección de valores de campos que proporcionan información sobre una entidad o una instancia de una relación. Los registros del mismo tipo se agrupan en tipos de registro.

Un tipo de relación padre-hijo es una relación 1:N entre dos tipos de registro. El tipo de registro del lado 1 se denomina registro padre, mientras que el del lado N se denomina registro hijo.

Por lo tanto, conceptualmente el esquema jerárquico define los datos en una estructura de árbol, donde cada registro tiene un único nodo raíz.

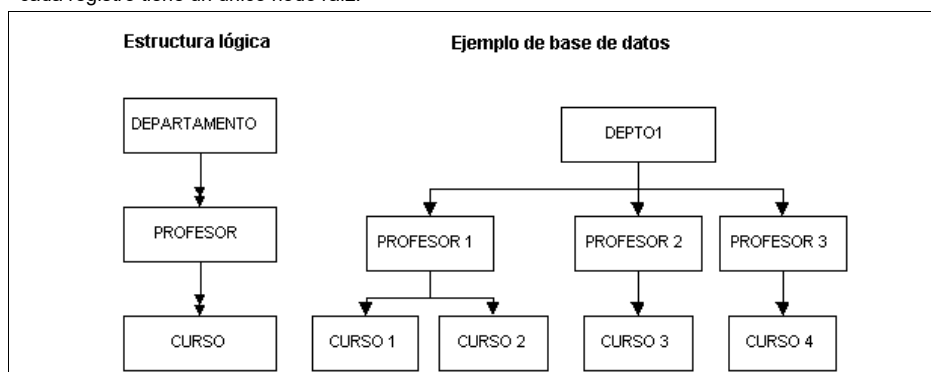


Figura 2: Ejemplo de modelo jerárquico.

La principal restricción de integridad que define el modelo jerárquico consiste en que ninguna ocurrencia de registro, a excepción del registro raíz, puede existir si no está relacionada con ninguna ocurrencia de registro padre. Esto tiene las siguientes implicaciones:

- Ningún registro hijo puede insertarse si no está enlazado a un registro padre.
- Un registro hijo se puede eliminar independientemente de su padre; pero la eliminación de un registro padre causa automáticamente la eliminación de todos sus registros hijo y sus descendientes.

La implementación del modelo jerárquico se realiza utilizando punteros. La estructura física, organización y tipos de punteros varía según los productos, e incluso el mismo producto proporciona diferentes organizaciones físicas, con el fin de conseguir una mayor eficiencia en el diseño del modelo físico.

El modelo jerárquico es muy eficiente en accesos asociados con la jerarquía del árbol, en cambio presenta importantes inconvenientes como:

- Su rigidez e incapacidad de representar sin redundancias ciertas estructuras muy difundidas como las relaciones reflexivas y N:M.
La poca flexibilidad de este modelo puede obligar a la introducción de redundancias.
- Un pequeño cambio en el programa puede suponer grandes modificaciones en la base de datos.
- Las actualizaciones pueden originar problemas, debido a la restricción de integridad comentada anteriormente.

Los SGBD basados en modelo jerárquicos suelen facilitar instrumentos que los dotan de mayor flexibilidad para representar estructuras no jerárquicas, como el concepto de padre virtual que implementa el SGBD IMS.

3.2.2 Modelo en red (CODASYL).

El modelo y lenguaje en red originales fueron presentados en el informe de 1971 del grupo de trabajo sobre bases de datos de CODASYL.

En este modelo existen dos estructuras de datos básicas: registros y conjuntos. Los datos se almacenan en registros. Cada registro consiste en un grupo de valores de datos relacionados entre sí.

CONTENIDO

Los registros se clasifican en tipos de registro, cada uno de los cuales describe la estructura de un grupo de registros que almacenan el mismo tipo de información.

Un tipo de conjunto es una descripción de un vínculo 1:N entre dos tipos de registro. Cada definición de tipo de conjunto consta de tres elementos básicos:

- Un nombre.
- Un tipo de registro propietario
- Un tipo de registro miembro.

En una base de datos habrá muchas ocurrencias de conjuntos (o instancias de conjuntos) que corresponderán a un tipo de conjunto. Cada ocurrencia relaciona un registro del tipo de registro propietario con el conjunto de registros del tipo de registro miembro. Así, cada ocurrencia de conjunto se compone de:

- Un registro propietario del tipo de registros propietarios.
- Una serie de registros miembro relacionados (cero o más) del tipo de registros miembro.

A modo de ejemplo, podríamos definir un tipo de conjunto llamado DPTO_PROF con el tipo de registro DEPARTAMENTO como tipo de registro propietario; y un tipo de registro PROFESOR como tipo de registro miembro.

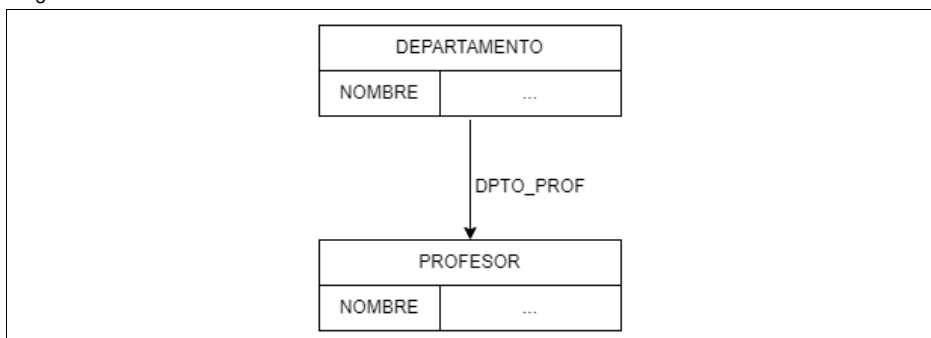


Figura 3: Tipo de conjunto DPTO_PROF

Algunas de las restricciones de integridad más destacadas de este modelo son las siguientes:

- Un registro del tipo de registro miembro no puede existir en más de una ocurrencia de conjunto de un tipo de conjunto concreto. Esto mantiene la restricción de que un tipo de conjunto representa un vínculo 1:N.
- Una ocurrencia de conjunto puede identificarse por el registro propietario o bien por cualquiera de los registros miembro.
- Los registros miembro de un ejemplar de conjunto están ordenados.

Aunque este modelo permite más flexibilidad que el modelo jerárquico, y en algunos casos se adapta muy bien a algunos tipos de transacciones, se considera superado por otros modelos, como el relacional, o subsumido en parte por modelos más modernos, como el orientado a objetos.

3.2.3 Modelo relacional.

El modelo relacional de bases de datos se basa en la lógica de predicados y en la teoría de conjuntos. Fue propuesto por Codd en 1970. Su idea fundamental es el uso de relaciones que pueden vincularse entre sí. La mayoría de las veces se conceptualiza de una manera más fácil de imaginar, pensando en cada relación como si fuese una tabla que está compuesta por registros (cada fila de la tabla sería un registro o "tupla") y columnas (también llamadas "atributos"). Las tablas pueden vincularse entre sí por un campo en común.

Sus principales ventajas son el fácil acceso, la seguridad y la estandarización. Una de sus desventajas más importantes es que el rendimiento disminuye cuando se manejan grandes cantidades de datos.

Los conceptos básicos del modelo relacional son los siguientes:

- Relación. Base del modelo, es un conjunto de datos que representa una realidad. A nivel práctico se le denomina tabla.
- Tupla. Cada fila de una tabla.

- Atributo. Un atributo es el nombre de una columna de una tabla.
- Grado de una relación. Número de atributos de una relación (columnas de una tabla).
- Dominio de un atributo. Conjunto de valores que puede tomar un atributo.
- Clave candidata y clave primaria. Se denomina clave candidata a todo atributo o conjunto de atributos que identifica de manera unívoca a la tupla. De entre las claves candidatas se elige una que será la clave primaria de la relación.
- Clave ajena. Una clave ajena es un atributo presente en una tabla que hace referencia a un atributo presente en otra tabla.
Valor nulo: utilizado para representar información desconocida, inaplicable, inexistente, no válida, no proporcionada, indefinida.
- Restricciones de integridad. Son normas que aseguran la correctitud y completitud de la información:
 - Integridad de dominio: La integridad de dominio es la validez de las restricciones que debe cumplir una determinada columna de la tabla.
 - Integridad de entidad: establece que la clave primaria de una tabla debe tener un valor único para cada fila de la tabla; si no, la base de datos perderá su integridad
 - Integridad referencial: asegura la integridad entre las claves ajenas y primarias.
 - Integridad definida por el usuario. Comprende las reglas definidas por el usuario para cumplir con sus requisitos específicos.

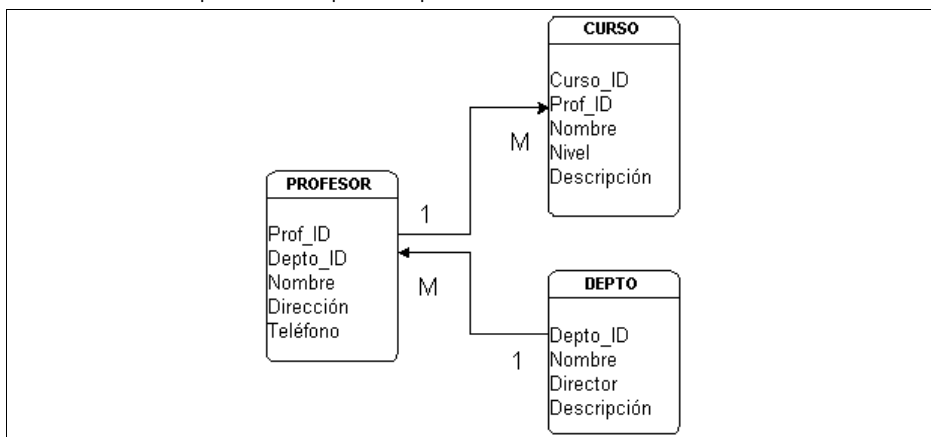


Figura 4: Ejemplo de disposición relacional.

Para poder determinar con certeza si un gestor era fiel al modelo relacional, Codd publicó en 1984 un conjunto de reglas que definen las características esperadas de un sistema relacional, siendo necesario que un gestor cumpla la primera regla (denominada regla cero) y al menos seis de las restantes para que pueda considerarse relacional (son 13 reglas, de la 0 a la 12).

El modelo relacional se estudia con más profundidad en otro tema del temario.

3.2.4 Modelo orientado a objetos y modelo objeto-relacional.

La característica de las bases de datos orientada a objetos es que almacenan objetos. Aparecieron en los años 80 del siglo XX con el objetivo de que se pudieran integrar directamente con aplicaciones desarrolladas con lenguajes orientados a objetos. El modelo fue estandarizado a principios de los 90 por el *Object Database Management Group* (ODMG). En el año 2000 se publicó la última versión del estándar, la 3.0.

Los objetos de la base de datos pueden tener propiedades, operaciones (métodos) y relaciones. Se categorizan en tipos. Cada tipo tiene un dominio específico compartido por todos los objetos de ese tipo. También existen los conceptos de herencia y polimorfismo. En lugar de utilizar SQL, en este modelo se utiliza el lenguaje de definición de objetos (ODL, *Object Definition Language*) y el de consulta de objetos (OQL, *Object Query Language*).

En relación con las bases de datos orientadas a objetos, ha habido históricamente dos corrientes. El enfoque purista de Atkinson (finales de los 80) propugna que se debe seguir un modelo orientado a objetos puro. Por otro lado, el manifiesto de Stonebraker (1990) defiende que lo ideal es implementar sistemas relacionales extendidos que sean capaces de soportar los conceptos principales de orientación a objetos.

Las bases de datos orientadas a objetos puras no han tenido mucho éxito. La postura que han seguido los principales fabricantes de bases de datos ha sido la de extender el modelo relacional. Es por ello por lo que surgió el modelo objeto-relacional, que implementan muchos sistemas conocidos como Oracle o PostgreSQL. Mantienen el esquema de tablas entre las que se establecen relaciones, pero permitiendo atributos de tipos complejos denominados tipos abstractos de datos (en Oracle, por ejemplo, se definen con la sentencia CREATE TYPE).

Uno de los modelos objeto-relacional más extendidos es el de las bases de datos espaciales. Son bases de datos relacionales que incorporan tipos predefinidos de datos (punto, línea, polígono), los cuales tienen ciertas propiedades (longitud, área, sistema de referencia, entre otros) y métodos propios (por ejemplo, proyectar); es decir, son asimilables a un objeto.

Las ventajas del modelo de base de datos orientada a objetos son la encapsulación, la reusabilidad, la extensibilidad y la mejor correspondencia entre paradigma de programación y paradigma de base de datos, ya que en la actualidad casi todo se desarrolla en orientación a objetos. Uno de los mayores inconvenientes es la falta de una teoría matemática que le sirva de base (como sí tiene el modelo relacional), además de que es difícil optimizar el rendimiento de las consultas manteniendo el requisito de encapsulación.

3.2.5 Bases de datos documentales y bases de datos multimedia.

Bases de datos documentales

En la actualidad, el término bases de datos documentales puede tener dos acepciones: una desde el punto de vista de las bases de datos NoSQL (como *MongoDB*); y otro desde el punto de vista de la archivística y la gestión documental. Nos centraremos ahora en este segundo enfoque, puesto que el primero se expone más adelante.

La gestión documental consiste en la gestión de grandes volúmenes de información no estructurada (texto, imágenes, gráficos, sonidos, etc.). Para ello, se utilizan los sistemas de gestión documental. Esos sistemas suelen contar, entre sus componentes, con un sistema gestor de bases de datos documentales, que están basados en sistemas de archivo y ficheros inversos. Los rasgos más característicos de un SGBD documental son:

- Capacidad para almacenar información textual de longitud grande y variable.
- Capacidad para recuperar con rapidez registros que responden a un criterio de búsqueda.
- Capacidad para realizar búsquedas multicriterio sobre ficheros inversos utilizando lógica booleana.
- Capacidad para administrar tesauros y diccionarios terminológicos.

Como ejemplos de sistemas de este tipo podemos citar: *BRS/Search*, *Inmagic* y *CD-Isis*.

Bases de datos multimedia

Los Sistemas de Bases de Datos Multimedia se caracterizan por almacenar, además de texto y otros datos discretos, información de audio y vídeo, permitiendo su manipulación y recuperación. Requieren una tecnología propia de almacenamiento y un sistema de ficheros.

Existen diferentes tipos de SGBD multimedia, como el modelo basado en añadir extensiones multimedia a un SGBD relacional. Otro modelo se basa en bases de datos orientadas a objetos multimedia. Por último, existen sistemas híbridos que combinan ambos modelos.

El almacenamiento es uno de los principales problemas en estos SGBD. La solución pasa básicamente por el uso de sistemas ópticos de almacenamiento masivo y la aplicación de modelos de almacenamiento distribuido. Por otro lado, la aplicación de mecanismos normalizados de compresión permite una transmisión y almacenamiento más eficaces.

La recuperación de información en estos SGBD ha de tener en cuenta el contenido multimedia de las imágenes, audio o vídeo. Esto implica análisis de contenido, extracción de características, modelo de contenido, indexación y consulta.

3.2.6 Modelos NoSQL.

En la actualidad, la tendencia más reciente en cuanto a bases de datos no relacionales es el grupo de modelos denominados NoSQL. Suele decirse que proviene del inglés *Not Only SQL* (no sólo SQL) y comenzó a utilizarse a finales de la década de los 90 del siglo XX, aunque su expansión ha sido muy posterior. Bajo el paraguas de NoSQL se encuentran una gran variedad de modelos de bases de datos cuyo punto en común es que no se basan en el modelo relacional y que utilizan otros lenguajes de consulta distintos a SQL, aunque algunos pueden usar SQL también.

Estas bases de datos empezaron a aparecer como soporte para las principales redes sociales de *Google*, *Twitter* y *Facebook*, las cuales tenían necesidad de procesar grandes volúmenes de datos en poco tiempo. Estas empresas encontraron algunos desafíos que las bases de datos relacionales no solucionaban bien. Estas últimas invierten bastantes recursos en mantener la coherencia; sin embargo, estas empresas preferían penalizar la coherencia si con ello se obtenía un rendimiento mucho mayor. De ahí surgieron los primeros sistemas NoSQL.

Las características principales de estos modelos son las siguientes:

- No usan SQL como lenguaje de consultas, o lo utilizan como un lenguaje de apoyo.
- No utilizan estructuras fijas como tablas para el almacenamiento de los datos.
- No suelen permitir operaciones JOIN, debido al coste en rendimiento.
- Como consecuencia del punto anterior, suelen fomentar la desnormalización de los datos e incluso la introducción de redundancias en la base de datos.
- Siguen una arquitectura distribuida.

En general se considera que existen cuatro grandes tipos: orientados a documentos, orientados a columnas, de clave-valor y en grafo.

3.2.6.1 Modelos orientados a documentos.

Los modelos orientados a documentos gestionan documentos, vistos como un conjunto de datos estructurados en algún lenguaje tipo XML, JSON o BSON. Son muy versátiles y las más conocidas son *MongoDB* y *Apache CouchDB*, de código abierto.

Un documento es el equivalente a un registro en una base de datos relacional. Se compone de campos de nombre y valor. Cada campo es una asociación entre un nombre y un valor y es similar a una columna en una base de datos relacional.

Una colección es un grupo de documentos y se corresponde con una tabla creada con cualquier otro SGBD relacional como Oracle o MySQL. No tiene una estructura predefinida.

En este modelo, una base de datos es un contenedor de colecciones, al igual que un SGBD relacional es un contenedor de tablas. Cada uno tiene su propio conjunto de archivos en el sistema de archivos.

3.2.6.2 Modelos orientados a columnas.

Las orientadas a columnas están optimizadas para realizar consultas sobre enormes cantidades de datos. Almacenando columnas de datos en lugar de registros o tablas. Las más conocidas son *Apache Cassandra* (altamente escalable, usada en *Facebook* y *Twitter*) y *Apache HBase* (del ecosistema *Hadoop*), ambas de código abierto.

Las bases de datos relacionales tienen un esquema establecido y funcionan como tablas de filas y columnas. Las bases de datos de columnas tienen un esquema similar, pero diferente. También tienen filas y columnas. Sin embargo, no están fijos dentro de una tabla, sino que tienen un esquema dinámico. Cada columna se almacena por separado. Si hay columnas similares (relacionadas), se unen en familias de columnas y luego las familias de columnas se almacenan por separado de otras familias de columnas.

La clave de fila es la primera columna de cada familia de columnas y sirve como identificador de una fila. Además, cada columna posterior tiene una clave de columna (nombre). Identifica columnas dentro de filas y, por lo tanto, permite la consulta de las columnas. El valor y la marca de tiempo vienen después de la clave de columna, dejando un rastro de cuándo se ingresaron o modificaron los datos.

El número de columnas pertenecientes a cada fila, o su nombre, puede variar. En otras palabras, no todas las columnas de una familia de columnas y, por lo tanto, una base de datos, tienen el mismo número de filas. De hecho, aunque compartan su nombre, cada columna está contenida en una fila y no se ejecuta en todas las filas.

Comentado [MLTM9]: SE hace una descripción más detallada de sistemas jerárquico por ejemplo que de los actuales.

Comentado [AS10R9]: Se añaden más detalles de cada modelo NoSQL.

3.2.6.3 Modelo clave-valor

El modelo clave-valor es el más sencillo. Simplemente guardan tuplas que contienen una clave y su valor. Son muy eficientes y no se necesita definir ningún esquema. *Redis* y *Memcached* son dos ejemplos, de código abierto. En muchas ocasiones se utilizan a modo de caché, debido a su rapidez.

La base de datos almacena los datos en pares clave/valor. La clave puede ser sintética o autogenerada, así como también puede poseer diversos formatos. La condición obligatoria que ha de tener dicha clave es que debe ser única. Los valores se construyen en una estructura simple, que acepta diferentes tipos de formatos. Pueden ser numéricos, JSON u otros.

Otras características importantes que las bases de datos clave-valor poseen es que clasifican y almacenan los datos en diccionarios. Son fáciles de escalar en sentido horizontal y brindan una alta velocidad en materia de consultas y modificación de datos.

3.2.6.4 Modelos de grafos.

Por último, el modelo de grafos utiliza nodos y aristas para representar los datos almacenados. Son muy útiles para guardar información con muchas relaciones. *Infinite Graph* (privativo) y *Neo4J* (con versión libre y versión privativa) son dos ejemplos.

Un grafo es una composición de un conjunto de objetos llamados nodos conectados a través de una serie de aristas. En los nodos están conformados por información y las aristas que conectan los nodos nos permiten entender las relaciones que existen entre los nodos.

Los grafos pueden tener relaciones dirigidas o no, multiplicidad de relaciones entre nodos y distinta aridad de relaciones. Hay que destacar que, aunque las bases de datos en grafos son recientes, la teoría en la que se basa está consolidada.

Estos modelos usan grafos que permiten representar interacciones complejas entre datos. La base de datos carece de esquemas, lo que permite una gran flexibilidad. También admite las relaciones que se generan en una base de datos relacional.

Las bases de datos de grafos gozan de un excepcional rendimiento a la hora de manejar volúmenes grandes de datos. Ofrecen una diversidad de métodos de analíticos y de consultas; convirtiéndose así en una de las bases de datos con mayor potencialidad del universo NoSQL.

3.2.6.5 Ventajas e inconvenientes de NoSQL.

Entre las principales ventajas de los modelos NoSQL cabe destacar que se pueden ejecutar en máquinas de pocos recursos; pueden manejar enormes cantidades de datos; tienen muy buena escalabilidad horizontal; y no generan cuellos de botella.

Respecto a los inconvenientes, algunas de las críticas más frecuentes al modelo NoSQL son su falta de rigurosidad y robustez, así como su penalización de la consistencia, en comparación con las bases de datos relacionales. A la hora de plantearse utilizar estas bases de datos hay que tener muy claro que fueron diseñadas precisamente para priorizar el rendimiento sobre cualquier otra variable.

3.3 Arquitecturas de los SGBD.

La arquitectura de un SGBD se puede considerar desde enfoques o puntos de vista diferentes, que se expondrán en los siguientes subapartados.

3.3.1 Arquitectura del sistema.

La arquitectura del sistema especifica cómo se organiza la distribución del procesamiento del SGBD en el sistema informático. La arquitectura de los paquetes de SGBD ha evolucionado desde los primeros sistemas monolíticos, donde todo el paquete de software del SGBD era un sistema completamente integrado, a los actuales paquetes de SGBD que tienen un diseño modular, según la arquitectura cliente/servidor. Esta evolución refleja la tendencia en computación, donde los grandes computadores centralizados han ido siendo reemplazados por cientos de estaciones de trabajo y computadores personales distribuidos y conectados mediante redes.

3.3.1.1 SGBD centralizado.

Inicialmente, las arquitecturas para SGBD siguieron tendencias similares a las de las arquitecturas de sistemas de computadores. Las primeras arquitecturas empleaban macrocomputadoras para proporcionar el procesamiento principal a todas las funciones del sistema, incluyendo programas de aplicación de usuario, programas de interfaz de usuario, así como toda la funcionalidad del SGBD. La razón era que la mayoría de los usuarios accedía a tales sistemas a través de terminales que no tenían poder de procesamiento. Todo el procesamiento se realizaba de forma remota.

Debido al progresivo descenso de los precios del hardware, muchos usuarios comenzaron a sustituir sus terminales por ordenadores personales y estaciones de trabajo. En un período inicial, los SGBD siguieron funcionando de forma centralizada, con la ejecución del programa, el procesamiento de la interfaz y todas las funciones se realizaban en una única máquina. Posteriormente, los SGBD comenzaron gradualmente a explotar la potencia del procesamiento disponible del usuario, lo que llevó a las arquitecturas SGBD cliente/servidor.

3.3.1.2 SGBD cliente/servidor.

La arquitectura cliente/servidor se creó para manejar los entornos de computación en los que un gran número de ordenadores personales, estaciones de trabajo, servidores de ficheros, impresoras, servidores de bases de datos, servidores web y otros equipos están interconectados a través de una red. Generalmente, el software del servidor es el responsable de la gestión de los datos y la gestión de transacciones. Por otro lado, el software del cliente se encarga de los programas de aplicación y la interfaz de usuario.

3.3.2 Arquitectura de distribución de datos.

Los datos de una base de datos pueden implantarse en una única ubicación o bien distribuirse entre distintas localizaciones. Una base de datos centralizada se almacena en una sola ubicación física y los usuarios de esta trabajan normalmente de manera remota, accediendo a ella a través de una red de comunicaciones.

Una ventaja de las bases de datos centralizadas es la capacidad de acceder a toda la información en un solo lugar. Las búsquedas en la base de datos pueden ser rápidas, la información es más fácil de organizar y de proteger. En cuanto a los inconvenientes, una base de datos centralizada tiende a crear cuellos de botella si varios usuarios necesitan acceder a ella y sus necesidades son sustanciales. También puede ser muy vulnerable si no se tiene una correcta política de copia de seguridad.

Por su parte, una base de datos distribuida es una colección de múltiples bases de datos interrelacionadas lógicamente, distribuidas por una red de ordenadores. Un sistema gestor de bases de datos distribuidas (SGBDD) maneja estas bases de datos haciendo la distribución transparente al usuario. La distribución conlleva un incremento de la complejidad en el diseño, implementación y gestión del sistema. Además de las funciones de un SGBD centralizado, uno distribuido ha de proporcionar un gestor de consultas distribuidas, un gestor de transacciones distribuidas, un gestor de datos replicados, así como un gestor de recuperación distribuido.

A la hora de distribuir los datos, existen dos estrategias principales: la fragmentación y la replicación. La fragmentación consiste en dividir los datos en base a algún criterio y repartirlos en los nodos, sin replicación. Puede ser horizontal (división por tuplas), vertical (división por atributos o columnas) o mixta.

La replicación consiste en, una vez fragmentados los datos de alguna manera, distribuir tanto los fragmentos como copias de estos. Se pueden seguir diversas estrategias de replicación, desde el extremo de la replicación total (toda la base de datos se encuentra replicada en todos los nodos) hasta el extremo de tener sólo fragmentación sin replicación. La replicación mejora la disponibilidad, pero penaliza el rendimiento de inserciones y actualizaciones de datos.

Los SGBD distribuidos han de ofrecer transparencia de distribución (ocultar detalles de dónde se almacena cada dato), transparencia de localización (el usuario no necesita conocer la localización de un dato para acceder a él) y transparencia de fragmentación (el usuario no necesita conocer la existencia de distintos fragmentos de datos).

Las ventajas de las bases de datos distribuidas son el aumento de la disponibilidad, fiabilidad y eficiencia en el acceso a la información; la fácil escalabilidad horizontal; y el rendimiento proporcionado por la localización de los datos. Por otro lado, los inconvenientes son la mayor dificultad en el control

de acceso a los datos y de la seguridad; la dependencia de la red; y el coste de implementación puede ser alto. Hay que destacar también el teorema CAP o conjetura de Brewer, que establece que respecto a las variables consistencia, disponibilidad y tolerancia a particionado, los sistemas distribuidos penalizan alguna de ellas en favor de las otras dos.

En un esquema de base de datos distribuida puede haber distintos grados de homogeneidad y autonomía local de los nodos. Si el sistema es heterogéneo y existe gran nivel de autonomía, nos encontraríamos en un esquema de bases de datos federadas.

Los sistemas de bases de datos federadas son un tipo de sistemas de bases de datos distribuidas, donde cada nodo de la red dispone de un SGBD propio, independiente y autónomo, que tiene sus propios datos, usuarios y transacciones locales. Sin embargo, a pesar de tratarse de nodos independientes, existe un esquema o vista global de federación y se proporciona a los usuarios una interfaz común de consulta, que se encarga de dirigir la petición al nodo federado que corresponda.

Se debe remarcar que una base de datos federada no es una base de datos única distribuida, más bien es una solución para acceder de forma homogénea a información depositada en distintas bases de datos. En la Administración Pública, por ejemplo, podría tratarse de bases de datos de distintas administraciones (local, autonómica, estatal, europea) que tratan sobre un tema en común (por ejemplo, información medioambiental). Estas bases de datos pueden federarse de manera que se ofrezca una interfaz común de consulta. La federación es útil también cuando se fusionan órganos administrativos.

Existen sistemas fuertemente acoplados (poseen un esquema conceptual global) y débilmente acoplados (no tienen un esquema conceptual global).

3.3.3 Arquitectura interna: Elementos del SGBD.

La arquitectura interna de un SGBD es el modo en el que el SGBD se estructura internamente en partes funcionales.

3.3.3.1 Lenguajes del SGBD.

Todos los SGBD ofrecen lenguajes e interfaces apropiadas para cada tipo de usuario: administradores, diseñadores, programadores de aplicaciones y usuarios finales.

Los lenguajes van a permitir al administrador de la base de datos especificar los datos que la componen, su estructura, las relaciones que existen entre ellos, las reglas de integridad, los controles de acceso, las características de tipo físico y las vistas externas de los usuarios. Los lenguajes del SGBD se clasifican en:

- Lenguaje de definición de datos (LDD o DDL, de *Data Definition Language*): se utiliza para especificar el esquema de la base de datos, las vistas de los usuarios y las estructuras de almacenamiento. Es el que define el esquema conceptual y el esquema interno. Lo utilizan los diseñadores y los administradores de la BD.
- Lenguaje de manipulación de datos (LMD o DML, de *Data Manipulation Language*): se utilizan para leer y actualizar los datos de la base de datos. Es el utilizado por los usuarios para realizar consultas, inserciones, eliminaciones y modificaciones. Los hay procedurales, en los que el usuario será normalmente un programador y especifica las operaciones de acceso a los datos llamando a los procedimientos necesarios. Estos lenguajes acceden a un registro y lo procesan. Las sentencias de un LMD procedural están embebidas en un lenguaje de alto nivel llamado anfitrión. Las bases de datos jerárquicas y en red utilizan estos LMD procedurales. No procedurales son los lenguajes declarativos. En muchos SGBD se pueden introducir interactivamente instrucciones del LMD desde un terminal, también pueden ir embebidas en un lenguaje de programación de alto nivel. Estos lenguajes permiten especificar los datos a obtener en una consulta, o los datos a modificar, mediante sentencias sencillas. Las BD relacionales utilizan lenguajes no procedurales como SQL (*Structured Query Language*) o QBE (*Query By Example*).
- Lenguaje de control de datos (LCD o DCL, de *Data Control Language*): Permiten conceder o suprimir privilegios a los usuarios, es decir, realiza el control del acceso a los datos. Con este lenguaje se establecen las vistas de los usuarios, así a cada usuario se le permite manipular únicamente el conjunto de datos que le interesan, y se le deniega el acceso a los datos que no necesita.
- Lenguajes de control de transacciones: Controlan los cambios realizados en los datos de la base de datos mediante instrucciones de manipulación de datos (DML). Permiten agrupar

varias instrucciones DML como si fuesen una única instrucción lógica de manipulación, de forma que o se realiza el grupo de instrucciones completo o se deshacen los cambios de todo el grupo (esto ocurriría en el caso de que simplemente una de las instrucciones DML del grupo no pudiera realizarse o generara algún error en su ejecución).

- La mayoría de los SGBD comerciales incluyen lenguajes de cuarta generación (4GL) que permiten al usuario desarrollar aplicaciones de forma fácil y rápida, también se les llama herramientas de desarrollo. Ejemplos de esto son las herramientas del SGBD Oracle: *SQL Forms* para la generación de formularios de pantalla y para interactuar con los datos; *SQL Reports* para generar informes de los datos contenidos en la base de datos; *PL/SQL* lenguaje para crear procedimientos que interactúen con los datos de la base de datos.

3.3.3.2 El diccionario de datos.

El diccionario de datos es el lugar donde se deposita información acerca de todos los datos que forman la base de datos. Es una guía en la que se describe la base de datos y los objetos que la forman.

El diccionario contiene las características lógicas de los sitios donde se almacenan los datos del sistema, incluyendo nombre, descripción, alias, contenido y organización. Identifica los procesos donde se emplean los datos y los sitios donde se necesita el acceso inmediato a la información.

El diccionario de datos proporciona información acerca de:

- La estructura lógica y física de la BD.
- Las definiciones de todos los objetos de la BD
- El espacio asignado y utilizado por los objetos.
- Los valores por defecto de las columnas de las tablas.
- Información acerca de las restricciones de integridad.
- Los privilegios y roles otorgados a los usuarios.
- Auditoría de información, como los accesos a los objetos.

Un diccionario de datos debe cumplir las siguientes características:

- Debe soportar las descripciones de los modelos conceptual, lógico, interno y externo de la BD.
- Debe estar integrado dentro del SGBD.
- Debe apoyar la transferencia eficiente de información al SGDB. La conexión entre los modelos interno y externo debe ser realizada en tiempo de ejecución.
- Debe comenzar con la reorganización de versiones de producción de la BD. Además, debe reflejar los cambios en la descripción de la BD. Cualquier cambio a la descripción de programas ha de ser reflejado automáticamente en la librería de descripción de programas con la ayuda del diccionario de datos.
- Debe estar almacenado en un medio de almacenamiento con acceso directo para la fácil recuperación de información.

3.3.3.3 Interfaces del SGBD.

Entre las interfaces amigables con el usuario que pueden ofrecer los SGBD están las siguientes:

- Interfaces para navegación basadas en menús. Presentan al usuario listas de operaciones denominadas menús, que guían al usuario para formular una solicitud.
- Interfaces basadas en formularios. Presentan un formulario a cada usuario.
- Interfaces gráficas de usuario. Sueñen presentar al usuario los esquemas en forma de diagrama.
- Interfaces de lenguaje natural. Aceptan solicitudes escritas en lenguaje natural y las procesan para traducirlas internamente.
- Interfaces para usuarios paramétricos. Se trata de interfaces para usuarios que siempre ejecutan unas determinadas operaciones, como los cajeros de un banco, por ejemplo.
- Interfaces para el DBA. Incluyen instrucciones privilegiadas que sólo el DBA puede ejecutar.

3.3.3.4 Módulos componentes.

Los componentes más destacados del SGBD son los siguientes:

- Un módulo gestor de datos almacenados del SGBD controla la información almacenada en el disco. Puede emplear servicios básicos del sistema operativo para transferir los datos de bajo

nivel entre el disco y la memoria principal. También controla otros aspectos de la transferencia de datos, como el manejo de los búferes en la memoria principal.

- El compilador de DDL procesa las definiciones de esquemas especificadas en el DDL y almacena las descripciones de los esquemas (metadatos) en el catálogo del SGBD.
- El procesador de base de datos en tiempo de ejecución se encarga de los accesos a la misma durante la ejecución. Recibe operaciones de consulta o de actualización y las ejecuta sobre la base de datos.
- El compilador de consultas maneja las consultas de alto nivel que se introducen interactivamente. Analiza la sintaxis y compila la consulta o la interpreta creando el código de acceso a la base de datos; y luego genera llamadas al procesador en tiempo de ejecución para ejecutar dicho código.
- El precompilador extrae instrucciones en DML de un programa de aplicación escrito en un lenguaje de programación anfitrión, que se envían al compilador de DML.
- El compilador de DML convierte las instrucciones DML en código objeto para el acceso a la base de datos.

3.3.3.5 Utilidades.

Las utilidades ayudan al DBA a gestionar el sistema. Suelen efectuar los siguientes tipos de funciones:

- Carga. Se utiliza una utilidad de carga para cargar ficheros de datos ya existentes (como ficheros de texto o secuenciales) en la base de datos.
- Copia de seguridad (*backup*). Las utilidades de respaldo crean una copia de seguridad de la base de datos. Esta copia puede servir para restaurar la base de datos en caso de un fallo catastrófico.
- Reorganización de ficheros. Esta utilidad sirve para pasar de una organización de los ficheros de la base de datos a otra con el fin de mejorar el rendimiento.
- Control del rendimiento. Las utilidades de este tipo supervisan la utilización de la base de datos y proporcionan datos estadísticos al DBA.

3.3.4 Arquitectura de referencia.

Por último, la arquitectura de referencia determina los niveles de abstracción de los datos en el SGBD. Suelen ser de dos o tres niveles. La de dos niveles, hoy en desuso, constaba de nivel físico y nivel lógico. La arquitectura de tres niveles más extendida es el modelo de referencia ANSI/SPARC, que se ha visto anteriormente.

3.4 Resumen del apartado.

Desde su aparición, los SGBD han ido evolucionando en cuanto a los modelos de datos que implementan y en cuanto a su arquitectura.

En relación con los modelos, los primeros fueron el modelo jerárquico y el modelo en red (CODASYL). Posteriormente, apareció el modelo relacional, el modelo de objetos, el modelo objeto-relacional y los modelos NoSQL. Actualmente los modelos más utilizados son el objeto-relacional y los modelos NoSQL.

Cuando hablamos de arquitectura del SGBD, distinguimos entre arquitectura del sistema (monolítico o cliente/servidor); arquitectura de datos (base de datos centralizada, distribuida, federada); y arquitectura interna del SGBD, esto es, los elementos que lo componen: los lenguajes (DDL, DML, DCL, entre otros); el diccionario de datos; las interfaces; los módulos principales (gestor de datos almacenados, compilador de DDL, procesador en tiempo de ejecución, compilador de consultas, precompilador y compilador de DML); y las utilidades. También suele hablarse de la arquitectura de referencia, que especifica los niveles de abstracción que implementa el SGBD, siendo actualmente la arquitectura ANSI la más extendida.

4 Gestor transaccional. Control de concurrencia. Bloqueos. Recuperación de errores. Integridad.

4.1 Gestor transaccional.

El gestor transaccional es un componente del SGBD que procesa información descomponiéndola de forma unitaria en operaciones indivisibles, llamadas transacciones. La transacción es la unidad lógica de procesamiento de la base de datos, que incluye una o más operaciones, que pueden ser de inserción, eliminación, modificación o recuperación. Una unidad lógica de trabajo simple debe exhibir las denominadas cuatro propiedades ACID¹, del inglés *Atomicity, Consistency, Isolation, Durability* (atomicidad, consistencia, aislamiento y persistencia). Cada transacción debe finalizar de forma correcta o incorrecta como una unidad completa. No puede acabar en un estado intermedio.

Propiedades ACID de las transacciones:

- Atomicidad: Una transacción debe ser una unidad atómica de trabajo: o todas sus operaciones se llevan a cabo o no se realiza ninguna de ellas. Lo que conlleva la capacidad de poder deshacer algunas operaciones.
- Consistencia: Cuando termina, una transacción debe dejar la base de datos en un estado consistente. La verificación de la consistencia es responsabilidad del control semántico. Asegurar la consistencia es responsabilidad de los mecanismos de control de concurrencia.
- Aislamiento: Las modificaciones realizadas por una transacción deben aislarse de las modificaciones llevadas a cabo por otras posibles transacciones concurrentes. Una transacción debe ver los datos en el estado en el que estaban antes de que cualquier otra transacción concurrente los modificara, o bien los ve tras su modificación, pero nunca en un estado intermedio. El aislamiento es una propiedad de las transacciones por la cual una transacción ve en todo momento la base de datos en un estado consistente. Una transacción en ejecución no hace visibles sus datos a otras transacciones concurrentes hasta que no termina y hace permanentes sus cambios.
- Persistencia: Una vez que la transacción ha terminado con éxito, sus efectos deben hacerse permanentes en la base de datos. Las modificaciones deben persistir incluso en el caso de un fallo del sistema.

El SGBD garantiza que los resultados de las transacciones que han terminado sobrevivirán a posibles fallos posteriores del sistema. Esto se enlaza con el problema de la recuperación de la base de datos: como recuperar la base de datos a un estado consistente previo a la caída o fallo del sistema, donde se asegure la persistencia de las transacciones que han terminado y la cancelación de las transacciones no terminadas, de cara a garantizar la atomicidad.

Los programadores son los responsables de establecer el inicio y el final de cada transacción en puntos que hagan cumplir la consistencia lógica de los datos. El programador debe definir la secuencia de modificaciones de datos que dejen los datos en un estado consistente acorde con las reglas de negocio. En cualquier caso, el SGBD pone a disposición del administrador mecanismos para hacer cumplir esas reglas, de modo que la verificación de la consistencia lógica de los datos es responsabilidad de los mecanismos de control semántico del SGBD.

4.2 Control de concurrencia. Bloqueos.

Un SGBD ha de realizar un control de la concurrencia porque pueden surgir problemas cuando las transacciones se ejecutan de forma no controlada. Estos problemas han sido ampliamente estudiados y se clasifican en los siguientes grupos: lectura sucia, lectura no repetible y lectura fantasma.

4.2.1 Lectura sucia.

Una transacción modifica el valor de un dato y otra transacción lee ese valor actualizado. Posteriormente la primera transacción aborta, por lo que el valor modificado deja de existir.

¹ No confundir las propiedades ACID de las transacciones con las propiedades ACID relativas a la seguridad de la información (Autenticidad, Confidencialidad, Integridad, Disponibilidad).

CONTENIDO

En el ejemplo que se muestra en la Figura 5 vemos como el usuario 2 ha recibido un dato incorrecto, proveniente de una modificación realizada por el usuario 1 que, finalmente, abortó la transacción.

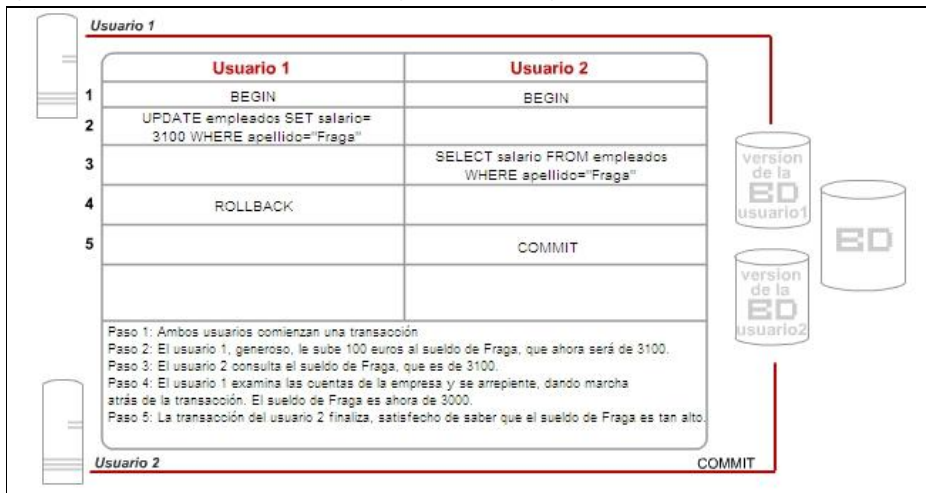


Figura 5: Ejemplo de lectura sucia.

4.2.2 Modificación perdida.

Algunos autores identifican este problema como un problema independiente, pero en realidad se trata de un caso particular de la lectura sucia. La modificación perdida surge cuando dos o más transacciones acceden a la misma fila y modifican su valor basándose en el valor original de la misma. Como resultado, una de las modificaciones se considera perdida.

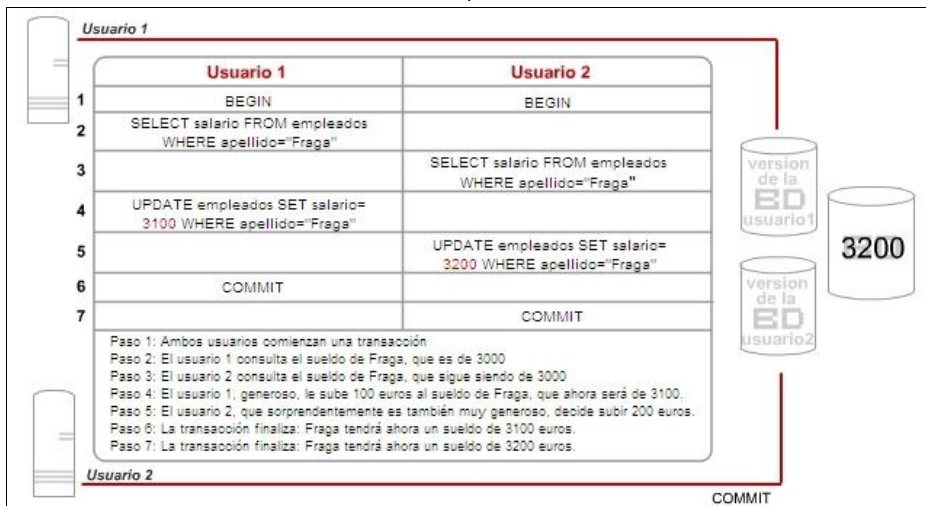


Figura 6: Ejemplo de modificación perdida.

4.2.3 Lectura no repetible.

Una transacción lee el valor de un dato dos veces consecutivas, pero entre esas dos lecturas, otra transacción modifica ese mismo valor. En el ejemplo que se muestra en la Figura 7 vemos como el usuario 1, dentro de la misma transacción, ha obtenido dos resultados distintos para el mismo dato.

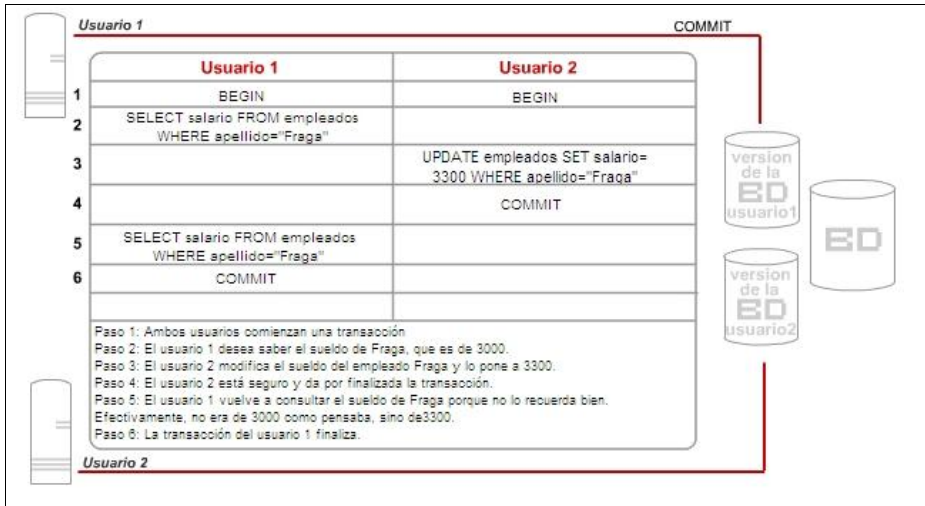


Figura 7: Ejemplo de lectura no repetible.

4.2.4 Lectura fantasma.

Durante una transacción, se ejecutan dos consultas idénticas sobre un conjunto de datos, y los resultados de la segunda no son iguales a los de la primera, porque otra transacción ha realizado modificaciones. La diferencia con respecto a la lectura no repetible es que en este caso trabajamos con conjuntos de datos y no con un dato específico.



Figura 8: Ejemplo de lectura fantasma.

4.2.5 Bloqueos.

Para controlar los problemas de concurrencia se implementan los bloqueos. El SGBD usa el bloqueo para asegurar la integridad de las transacciones y la consistencia de la base de datos. El bloqueo garantiza el aislamiento de las transacciones impidiendo a los usuarios leer datos que están siendo modificados por otros usuarios e impide que múltiples usuarios puedan modificar el mismo dato al mismo tiempo. Si no se usa el bloqueo, los datos de la base de datos pueden volverse inconsistentes y las consultas realizadas sobre esos datos producir resultados inesperados.

Un bloqueo es una información del tipo de acceso que se permite a un elemento. El SGBD impone los bloqueos necesarios en cada momento. El gestor de acceso a los datos implementa las restricciones de acceso. En algunos sistemas se permite que el usuario pueda indicar el bloqueo más adecuado.

El gestor de bloqueos del SGBD gestiona la asignación de bloqueos a las transacciones. Existen tres tipos de bloqueo principales:

- Bloqueo de lectura: sólo permite lectura.
- Bloqueo de escritura: permite lectura y escritura.
- Bloqueo de rango: bloquea un rango (conjunto) de elementos.

Además de los tipos de bloqueo, existen varios modos de bloqueo. El modo indica la forma en que se bloquea un elemento:

- Modo compartido: para operaciones sólo de lectura. Se permiten lecturas concurrentes, pero ninguna actualización.
- Modo actualización: para operaciones que pueden escribir. Sólo se permite que una transacción adquiera este bloqueo. Si la transacción modifica datos, se convierte en exclusivo, en caso contrario en compartido.
- Modo exclusivo: para operaciones que escriben datos. Sólo se permite que una transacción adquiera este bloqueo.
- Modo intención: se usa para establecer una jerarquía de bloqueo. Por ejemplo, si una transacción necesita bloqueo exclusivo y varias transacciones tienen bloqueo de intención, no se concede el exclusivo.
- Modo esquema: para operaciones de definición de datos (DDL, *Data Definition Language*).
- Modo actualización masiva: En operaciones de actualización masiva.

El gestor de bloqueos debe disponer de mecanismos para gestionar los bloqueos indefinidos (interbloqueos, *livelock*, *deadlock*), que suceden cuando una transacción espera indefinidamente por un bloqueo que no se le llega a conceder. Algunos mecanismos pueden ser el uso de colas FIFO, política de conceder o todos los bloqueos o ninguno a una transacción, etc.

4.2.5.1 Niveles de aislamiento.

A partir de los bloqueos, los SGBD incorporan distintos niveles de aislamiento.

- El nivel 0, llamado lectura no confirmada (*read uncommitted*), no incorpora ningún bloqueo, por lo que no evita ninguno de los problemas anteriores. Dado que no bloquea nada no aísla, es nivel más rápido.
- El primer nivel es el de lectura confirmada (*read committed*), que incluye el bloqueo de lectura, evitando así el problema de la lectura sucia.
- El segundo nivel, lectura repetible (*repeatable read*), tiene los bloqueos de lectura y escritura; y evita los problemas de lectura sucia y lectura no repetible.
- Por último, el nivel 4, secuenciable (*serializable*), incluye los bloqueos de lectura, escritura y rango, evitando los 3 problemas de concurrencia descritos. Además de reducir el rendimiento, puede provocar la aparición de interbloqueos (un problema de concurrencia que provoca que una transacción no pueda finalizar nunca debido a que otra lo está bloqueando indefinidamente).

4.3 Recuperación de errores.

El SGBD debe dejar la información de la base de datos en un estado correcto tras un error que haya podido dejar a la base de datos en un estado inconsistente. Los errores pueden ser debidos a acciones

CONTENIDO

de usuario, errores de programación, bloqueos mortales, fallos del sistema, fallos de disco y fallos físicos o catastróficos.

La recuperación de errores se basa en dos elementos fundamentales: las copias de seguridad y la bitácora, un fichero donde se van almacenando las operaciones efectuadas como parte de las transacciones. La estrategia de recuperación representativa es una de las más utilizadas y consiste en lo siguiente:

- En fallos de disco o fallos físicos: Restaurar la copia de seguridad de la BD y reconstruir el estado más actual, rehaciendo las operaciones de transacciones confirmadas hasta el momento del fallo, a partir de la bitácora.
- En otros fallos: Invertir modificaciones que provocaron la inconsistencia. Deshacer algunas operaciones a partir de la bitácora. Si es necesario, asegurar cambios correctos, rehaciendo algunas otras operaciones.

Para implementar la estrategia, hay que elegir una técnica de recuperación. Las dos principales son las siguientes:

- Técnica basada en la actualización diferida. Ninguna transacción modifica la base de datos antes de llegar a su punto de confirmación. Se difiere la consolidación de cambios realizados. Si el fallo ocurre antes de alcanzar el punto de confirmación, no es necesario deshacer sus operaciones. En caso contrario, sí.
- Técnica basada en la actualización inmediata. Una transacción puede modificar la base de datos antes de llegar a su punto de confirmación. Algunos cambios realizados pueden consolidarse en disco antes de confirmarse. Si el fallo ocurre antes de alcanzar el punto de confirmación, es necesario deshacer sus operaciones. Si el fallo ocurre después de alcanzar su punto de confirmación, es necesario rehacer sus operaciones.

4.4 Integridad.

El término integridad de datos se refiere a la correctitud y completitud de la información en una base de datos. El SGBD debe preservar la integridad de sus datos almacenados en la mayor medida posible, asegurándose del cumplimiento de las restricciones de integridad del modelo y del usuario.

Los SGBD deben proporcionar la forma de definir las restricciones de integridad de usuario de una base de datos y una vez definida, debe velar por su cumplimiento. Las reglas de integridad del modelo, en cambio, no se deben definir para cada base de datos concreta, porque se consideran preestablecidas para todas las bases de datos de un modelo. Un SGBD de un modelo determinado debe velar por el cumplimiento de las reglas de integridad preestablecidas por su modelo.

El administrador puede adoptar las siguientes políticas para asegurar la integridad:

- Limitar el acceso a datos y cambiar los permisos para evitar modificaciones no autorizadas.
- Mantener una copia de seguridad periódica de los datos.
- Utilizar registros para monitorizar cuando se añaden, alteran o borran datos.
- Hacer auditorías internas para garantizar que la información está actualizada.

4.5 Resumen del apartado.

En este apartado se han expuesto someramente los principales mecanismos con los que debe contar el SGBD para garantizar lo siguiente:

- La gestión de las transacciones, de cara a asegurar su atomicidad, consistencia, aislamiento y persistencia (las cuatro propiedades ACID de las transacciones -*Atomicity, Consistency, Isolation, Durability*-).
- El control de la concurrencia, de manera que cuando haya varias transacciones concurrentes se garantice la consistencia de la base de datos en todo momento. Se han de evitar los problemas de lectura sucia, modificación perdida, lectura no repetible y lectura fantasma. Para ello, el SGBD implementará distintos tipos (lectura, escritura, rango) y modos (exclusivo, compartido, etc.) de bloqueo. Según los bloqueos que se implementen, el SGBD garantizará un nivel concreto de aislamiento, habiendo cuatro niveles definidos: lectura no confirmada, lectura confirmada, lectura repetible y secuenciable.

Comentado [MLTM11]: Sería interesante un cuadro resumen, en el que se ponga el tipos de problemas y niveles de aislamiento

Comentado [AS12R11]: Se añada tabla

CONTENIDO

A continuación, se muestra un cuadro resumen de los niveles de aislamiento (en la primera columna) y los problemas de concurrencia conocidos, indicando qué problemas soluciona cada nivel.

	Lectura sucia y/o modificación perdida	Lectura no repetible	Lectura fantasma
Lecturas no confirmadas	Puede pasar	Puede pasar	Puede pasar
Lecturas confirmadas	No puede pasar	Puede pasar	Puede pasar
Lecturas repetibles	No puede pasar	No puede pasar	Puede pasar
Secuenciable (serializable)	No puede pasar	No puede pasar	No puede pasar

- La recuperación de errores, que se basa en dos elementos fundamentales: las copias de seguridad y la bitácora. Existen dos estrategias principales: actualización diferida y actualización inmediata.
- La integridad. El SGBD debe garantizar la correctitud y completitud de la información en una base de datos. Existen dos tipos de reglas de integridad: del modelo y de usuario.

5 Principales SGBD utilizados en la Administración.

Uno de los SGBD más utilizados en la Administración es *Oracle* (software propietario), que actualmente sigue el modelo objeto-relacional, ya que implementa el modelo relacional, pero incorpora extensiones para crear y manejar objetos. Otro de los SGBD que se utilizan en la Administración es *PostgreSQL* (de código abierto) que, a pesar de tener un uso mucho menor, es ampliamente utilizado en un sector concreto: los sistemas de información geográfica (SIG). Otras de las soluciones utilizadas en algunos proyectos concretos son *MySQL/MariaDB*; *Informix*; y *Microsoft Access*, utilizado en algunos organismos para pequeñas bases de datos operativas o de apoyo; entre otras. Cabe destacar también que se están empezando a utilizar en algunos proyectos bases de datos NoSQL como *MongoDB*.

En este apartado se expondrán someramente las características de los tres SGBD principales: *Oracle*, *PostgreSQL* y *MySQL/MariaDB*.

5.1 Oracle.

5.1.1 Conceptos básicos.

El SGBD Oracle empezó a finales de los 70. Es software de tipo privativo (no libre) y actualmente implementa un modelo de base de datos objeto-relacional. Cuenta con diversas ediciones, entre las que cabe destacar la *Enterprise Edition* (para manejar grandes volúmenes de datos), la *Standard Edition* (para un tamaño medio, con instalación menos compleja que la Enterprise); y la *Express Edition*, gratuita pero muy limitada, pensada para proyectos muy pequeños.

Además de incorporar SQL como lenguaje, incluye un lenguaje propio, PL/SQL, de sintaxis similar a SQL, que ofrece extensiones de programación. Las principales ventajas de Oracle son su seguridad, rendimiento y adaptabilidad. Su mayor desventaja es sin duda su alto coste, tanto de adquisición como de mantenimiento.

5.1.2 Arquitectura.

La arquitectura de los SGBD Oracle, basada en el modelo cliente/servidor, consiste en una o más instancias de la base de datos para la gestión de los datos y uno o más procesos de escucha que conectan a los clientes de la base de datos con las instancias de la base de datos. El modelo sigue el patrón cliente/servidor.

Oracle mantiene dos estructuras principales de memoria: el área global de programa, *Program Global Area*, PGA; y el área global del sistema, *System Global Area* o también *Shared Global Area*, SGA. El PGA es la zona de memoria de cada proceso Oracle. No está compartida y contiene datos e información de control de un único proceso. El SGA es la zona de memoria en la que la base de datos guarda información sobre su estado. Esta estructura de memoria está disponible para todos los procesos, por eso se dice que está compartida.

El SGBD maneja una serie de estructuras lógicas. La básica es la tabla, donde se almacenan los datos. En un nivel superior están los *tablespaces*, que contienen distintos objetos relacionados (por ejemplo, todas las tablas de una aplicación). Dentro de un *tablespace*, el almacenamiento se organiza en segmentos que contienen una o varias extensiones (*extents*), siendo una extensión un conjunto de bloques Oracle contiguos.

En cuanto a las estructuras físicas, Oracle cuenta con una serie de ficheros que se establecen en el momento en que se crea la base de datos o en el momento en el que se crean *tablespaces*. Podemos clasificar los ficheros en los siguientes tipos: de datos (contiene datos como tablas, índices, etc.), temporales (vinculado a un *tablespace* temporal), de control (ficheros maestros) y redo log (contienen registro de cambios).

5.1.3 Administración, gestión y optimización.

Para iniciar y parar la instancia se usan los comandos STARTUP y SHUTDOWN, respectivamente.

Oracle dispone de una serie muy extensa de parámetros que sirven para configurar la base de datos y la instancia de Oracle. Los parámetros se almacenan en archivos especiales que son leídos por la instancia de Oracle antes de iniciarse. Algunos de los ficheros más importantes son:

- INIT.ORA, de texto, guarda los parámetros principales.

- SPFILE.ORA, binario, alternativa al anterior.
- LISTENER.ORA, configura el proceso de escucha en el servidor.
- TNSNAMES.ORA, configura la conexión en el cliente.
- SQLNET.ORA, necesario en el cliente y en el servidor, con parámetros para gestionar la conexión.

La configuración de la memoria se hace con los parámetros SGA_MAX (memoria máxima) y SGA_TARGET (memoria inicial). El DBA deberá gestionar el tamaño de segmentos, extensiones y bloques a través de algunos parámetros del fichero INIT.ORA. También se dispone de comandos para ver el tamaño de los *tablespaces*. El DBA también puede hacer uso del diccionario de datos para ver los elementos creados en la base de datos (tablas, vistas, disparadores, etc.). Por otro lado, el fichero de alerta (*alert log*) registra de forma cronológica los errores ocurridos en la base de datos.

Otra de las tareas del DBA es la gestión de usuarios. Durante la instalación de Oracle se crean dos cuentas administrativas (SYS y SYSTEM) y otras dos con permisos especiales para tareas de optimización y monitorización de la base de datos (SYSMAN y DBSMNP). A la hora de crear usuarios, se pueden utilizar diversas formas de autenticación: por sistema operativo, por fichero de contraseñas, por diccionario de datos, externa. Existen más de 100 privilegios que se le pueden asignar al usuario, de dos tipos: de sistema y de objeto.

Entre algunas de las recomendaciones para optimizar el SGBD, cabe destacar las siguientes: habilitar el *database buffer caché*, multiplexar los archivos de *redo log*, ajustar correctamente el tamaño del *redo log buffer*, habilitar la *caché shared pool* y activar el parámetro *db_block_checksum*.

Oracle ofrece diversas interfaces de gestión, como *SQL Developer*, *SQL*Plus*, *Oracle Enterprise Database Control*, *DataGrip*, entre otras.

5.2 PostgreSQL.

5.2.1 Conceptos básicos.

PostgreSQL, también conocido como *Postgres*, es una SGBD objeto-relacional, de código abierto, lanzado inicialmente en 1996. Está programado en C y su última versión es la 14.

El módulo independiente *PostGIS* añade soporte a datos espaciales. Garantiza una gran interoperabilidad, ya que ha sido certificado por el *Open Geospatial Consortium* (OGC). En cuanto a eficiencia, *PostGIS* supera a la extensión geográfica de *MySQL* e iguala a *Oracle Spatial*, con la ventaja, respecto a esta última, de una mejor interoperabilidad con muchos más productos.

Los beneficios de usar *PostgreSQL* son su alta concurrencia, escalabilidad, potencia, estabilidad, portabilidad y extensibilidad. Las dos grandes desventajas es la falta de soporte técnico oficial, al ser de código abierto; y el bajo rendimiento en entornos pequeños, ya que ha sido optimizado para manejar grandes volúmenes de datos.

5.2.2 Arquitectura.

PostgreSQL sigue una arquitectura cliente/servidor. Del lado del cliente se instala una librería llamada *libpq*, que es el proceso responsable de gestionar las comunicaciones con el servidor. En el servidor se ejecutan 2 procesos principales: *postmaster* y *postgres*. El *postmaster* es el responsable de aceptar las comunicaciones del cliente, autenticar y dar acceso. El *postgres* se encarga de la administración de las consultas a las bases de datos.

Por otro lado, el componente llamado *storage manager* es responsable de la administración general de almacenamiento de los datos, controla la gestión de archivos, bloqueos y control de la consistencia de la información.

Al igual que ocurría en *Oracle*, *PostgreSQL* utiliza tablas y *tablespaces*. La unidad mínima de almacenamiento en *PostgreSQL* se denomina página (*page*) o bloque (*block*). Un bloque en *PostgreSQL* ocupa siempre por defecto 8 KB. A nivel físico, *PostgreSQL* almacena los ficheros de datos en un directorio denominado *Data Directory*, donde se almacenan ficheros para las tablas y los índices. Las tablas muy grandes pueden tener más de un fichero.

5.2.3 Administración, gestión y optimización.

La instancia se maneja a través del comando *postgres*, utilizando alguna de las opciones *start*, *stop*, *restart* y *reload*.

Los principales ficheros de configuración de PostgreSQL son los siguientes:

- *postgresql.conf*, de texto, que contiene los parámetros principales, en formato nombre=valor. Se puede editar directamente o a través de un comando específico en el intérprete de comandos del sistema operativo.
- *pg_hba.conf*, de texto, que controla la autenticación de los clientes.

Por defecto, el usuario administrador de la base de datos se llama *postgres*. La forma de autenticar los usuarios puede ser contraseña, *trust* (apropiada para usuarios locales), basada en API de seguridad, basada en sistema operativo, con certificados, con RADIUS, entre otras.

Algunas recomendaciones para optimizar PostgreSQL son: utilizar comandos VACUUM y VACUUM FULL para recuperar espacio en disco; reconstruir índices; utilizar el comando ANALYZE para optimizar consultas.

5.3 MySQL / MariaDB.

5.3.1 Conceptos básicos.

MySQL es un SGBD relacional desarrollado bajo licencia dual: licencia libre (GPL) y licencia comercial de Oracle. Está considerada como la base de datos de código abierto más popular del mundo.

Tras la adquisición por parte de Oracle, una parte de los desarrolladores se escindió del proyecto, creando una rama llamada *MariaDB*, de código abierto. *MariaDB* se desarrolló con el objetivo de que pudiera ser la sustituta inmediata de *MySQL*.

Las principales ventajas de estos SGBD son que no necesitan altas prestaciones de computación para ofrecer un buen rendimiento; son fáciles de instalar y configurar; son muy portables y seguras. Entre sus inconvenientes, cabe mencionar que el rendimiento en operaciones de escritura es menor que en otros SGBD y que no son buenas opciones para bases de datos muy grandes.

5.3.2 Arquitectura.

La arquitectura de MySQL, basada en el modelo cliente/servidor, tiene como característica más notable el separar el motor de almacenamiento del resto de los componentes de la arquitectura. El diseño del gestor está preparado para que se pueda cambiar el gestor de almacenamiento.

Los componentes principales de la arquitectura son:

- Los conectores, para comunicarse con las aplicaciones clientes.
- El gestor de conexiones que permite a varios usuarios acceder a la base de datos.
- El procesador de consultas que se encarga de interpretar las instrucciones a ejecutar.
- El optimizador de consultas que permite verificar los permisos que posee un determinado usuario para trabajar en la base de datos.
- La caché de consultas que es un tipo de memoria auxiliar que le permite agilizar los procesos de consulta
- El control de concurrencia que se encarga de implementar bloqueos a los datos que están siendo modificados por un usuario para no crear conflicto con otros usuarios
- El gestor de transacciones y recuperación que genera respaldos a los datos cuando se efectúan diversos procesos de modificación.
- El motor de almacenamiento (*storage-engine*) se encarga de almacenar, manejar y recuperar información de una tabla. Los motores más conocidos son *MyISAM* e *InnoDB*.

5.3.3 Administración, gestión y optimización.

La instancia se maneja a través del comando *mysqld*, utilizando alguna de las opciones *start*, *stop* y *restart*. El archivo de configuración, *my.ini* o *my.cnf*, contiene las opciones de arranque. El usuario administrador de la base de datos por defecto es *root*. Los principales parámetros de optimización son el máximo de conexiones, el tamaño del buffer, el tamaño del caché de consultas, y el método de vaciado (*flush*) de datos.

6 Tendencias actuales.

Desde el año 2020, las empresas -y la Administración- se han vuelto más digitalizadas, siendo prácticamente imposible evitar el uso de bases de datos. Actualmente, es necesario disponer de la capacidad de obtener información a partir de los datos, siendo este aspecto clave para cualquier organización de éxito. Por lo tanto, cualquier arquitectura de información deben seguir siendo las bases de datos.

Entre las principales utilidades que proporciona una base de datos a una organización está agrupar todos los datos de esta en un mismo lugar, facilitar que se compartan los datos entre los diferentes departamentos, evitar la redundancia y mejorar la organización junto con realizar una interlocución adecuada con los clientes o usuarios. Así pues, si se gestiona adecuadamente una base de datos se podrán obtener ventajas como un aumento de la eficiencia, una simplificación del proceso, mayor seguridad de los datos almacenados y una mayor productividad, entre otras de sus ventajas.

En este apartado se van a describir diferentes tendencias de bases de datos que pueden ayudar a cualquier organización a mejorar la gestión de sus datos e información para prosperar de manera exitosa.

6.1 Procesamiento híbrido transaccional-analítico (HTAP, *hybrid transactional/analytical processing*).

Una organización convencional está acostumbrada a contar con dos infraestructuras de base de datos por separado, una para soportar las transacciones operacionales del día a día del negocio: OLTP (*On-Line Transactional Processing*), y otra para el análisis de datos para la toma de decisiones: OLAP (*On-Line Analytical Processing*).

Hoy en día, las aplicaciones empresariales generan más datos que nunca, por lo que hacer análisis de datos es pieza clave y fundamental para una organización que pretende ser competitiva; sin embargo, mantener dos ambientes de base de datos por separado es altamente costoso en presupuesto y esfuerzo humano.

El procesamiento híbrido transaccional-analítico "HTAP", por sus siglas en inglés (*Hybrid transactional/analytical processing*) es un término acuñado por Gartner en el 2014 que consiste en hacer análisis directamente sobre base de datos de producción en tiempo real.

Con HTAP se ejecutan cargas para análisis y transacciones en la misma base de datos en tiempo real, pero que a la vez sus operaciones no se ven afectadas; son muy escasos los motores de base de datos empresariales que cuentan con experiencia y madurez en tecnologías HTAP, *Informix Database* es una de ellas.

6.2 Bases de datos multimodelo.

Una base de datos multimodelo es una plataforma de procesamiento de datos que soporta múltiples modelos de datos, que definen los parámetros de cómo se organiza y acomoda la información en una base de datos. Ser capaz de incorporar múltiples modelos en una sola base de datos permite que los equipos de tecnología de la información (TI) y otros usuarios cumplan con diversos requisitos de aplicación sin necesidad de implementar diferentes sistemas de bases de datos.

A diferencia de los sistemas que se limitan únicamente al modelo de datos relacional líder del mercado, las bases de datos multimodelo no almacenan de forma uniforme los datos en una estructura de tablas basada en filas. Como resultado, pueden manejar diferentes formas de datos que no se ajustan al esquema rígido del modelo relacional, incluidos los tipos de datos no estructurados y semiestructurados. En algunos casos, sin embargo, el enfoque multimodelo puede limitar la integridad transaccional que utilizan los sistemas de gestión de bases de datos relacionales para mantener la exactitud y consistencia de los datos.

En cierta medida, el término base de datos multimodelo surgió al mismo tiempo que algunos proveedores NoSQL comenzaron a mezclar diferentes modelos de datos en sus ofertas de productos. Pero también había precursores de eso. Por ejemplo, *MarkLogic* admite modelos de datos de documentos, gráficos, relacionales y de otro tipo en su base de datos del mismo nombre; fundada en 2001. Además, *InterSystems* ofrece modelos de objetos y relacionales en su software Caché, una base de datos multidimensional que fue lanzada por primera vez en 1997.

Couchbase soporta modelos documentales y relacionales en su base de datos *Couchbase Server*. *DataStax*, que vende una versión de la base de datos de código abierto de *Apache Cassandra*, llamada *DataStax Enterprise*, combinó un modelo de datos de clave-valor con uno basado en columnas; y continuó agregando soporte de documentos y gráficos. Cada vez más, la búsqueda de texto se ve como otro modelo de datos que soportan algunos motores NoSQL; entre ellos se encuentra la base de datos *Riak* de *Basho*, que combina clave-valor, objetos y modelos de búsqueda.

6.3 Seguridad incorporada.

En cualquier aplicación, interfaz, producto y negocio es requisito necesario y fundamental la seguridad. En la digitalización que se está viviendo es fundamental mantener unos niveles altos de confianza y transparencia, donde se aseguren los datos.

Por ello, técnicas como el cifrado en movimiento con cifrado transparente de datos (TDE), la gestión flexible de claves, el cifrado en reposo, la trazabilidad y la auditoría, son necesarias y obligatorias para cualquier compañía.

Entre las mencionadas, destaca la técnica del TDE, empleada para cifrar los archivos de datos de *SQL Server* y *Azure SQL Database*. Además, permite cifrar los datos confidenciales en las bases de datos y proteger las claves que se emplean para cifrar datos con un certificado.

Este tipo de técnica para la seguridad ayuda con las regulaciones y pautas establecidas en la industria y en el ámbito de la ciberseguridad, de manera que si algún ciberdelincuente trata de robar archivos de datos no podría emplearlos porque necesitaría una clave de acceso a ellos. No obstante, es importante saber que TDE solo agrega una capa de protección para los datos en reposo. Deben protegerse también el sistema de archivos del sistema operativo y la capa de hardware.

6.4 Bases de datos inteligentes y autónomas.

Una plataforma de bases de datos autónomas (*Self-Driving Database Platform*, SDDP) proporciona capacidades de autoconducción a las bases de datos que se ejecutan en dicha plataforma, de esta manera se permite que las bases de datos perciban, tomen decisiones y realicen optimizaciones de forma independiente, con el fin de generar un servicio continuo.

Asimismo, las SDDP traen consigo otras capacidades como la administración de recursos físicos, la administración de la vida útil de una instancia, la seguridad y el escalado automático.

Se espera que, en el futuro, las bases de datos sean totalmente autónomas, permitiendo automatizar las hojas de ruta completamente en una base de datos. No obstante, resulta complejo la automatización absoluta cuando a menudo las compañías cambian de base de datos, dificultando la automatización de todo el proceso.

Actualmente, se puede emplear la inteligencia artificial para diferentes escenarios en el ámbito de las bases de datos como, por ejemplo, las cargas de trabajo o la aplicación en sistemas tradicionales para ajustar los parámetros y mejorar su escalabilidad, todo ello mediante el uso de algoritmos de aprendizaje automático que garanticen que sean sistemas seguros y se ejecuten sin problemas.

6.5 Conclusiones.

El mundo se está digitalizando a gran velocidad, así que la búsqueda de diferentes técnicas para tratar datos heterogéneos es un desafío para las organizaciones que buscan sacar el máximo partido a los avances más recientes de tecnología como la inteligencia artificial o el Internet de las cosas.

Los estudios de nuevas alternativas para las bases de datos en la nube buscan que el almacenamiento y la computación sean compatibles con una amplia variedad de interfaces de código abierto.

Con los nuevos tipos de bases de datos se admitirán diferentes modelos de tablas con columnas más anchas o series de tiempo, almacenamiento y análisis de datos estructurados, semiestructurados y no estructurados. Todo ello, tendrá como resultado un rendimiento mayor de los dispositivos inteligentes que requieran almacenamiento de datos de múltiples modelos, el análisis de metadatos, datos de series de tiempo y registros de dispositivos.

En el futuro, se buscará combinar la tecnología *blockchain* con el fin de proporcionar modificaciones inmutables en los sistemas de bases de datos, y tratar de aprovechar el nuevo hardware para cifrar datos y procesarlos sin necesidad de descifrarlos en las bases de datos

7 RESUMEN ESQUEMÁTICO.

- Sistemas de Gestión de Bases de Datos (SGBD). Definición.
 - Actores. El DBA (*Database Administrator*)
 - Ventajas del uso de los SGBD:
- Modelo de ANSI o ANSI/SPARC: definición. Tres niveles. Independencias de datos: física y lógica.
- Modelos de bases de datos: Concepto de modelo de bases de datos. Modelos más destacados:
 - Modelo jerárquico.
 - Modelo en red (CODASYL).
 - Modelo relacional.
 - Modelo orientado a objetos.
 - Bases de datos documentales.
 - Bases de datos multimedia.
 - Modelos NoSQL.
- Arquitecturas SGBD
 - Arquitectura del sistema: SGBD centralizado vs SGBD cliente/servidor
 - Arquitectura de distribución de datos: datos centralizados, datos distribuidos, datos federados.
 - Arquitectura interna del SGBD: elementos que lo componen:
 - Lenguajes: DDL, DML, DCL.
 - Diccionario de datos.
 - Interfaces.
 - Módulos componentes: gestor de datos almacenados; compilador de DDL; procesador en tiempo de ejecución; compilador de consultas; precompilador DML; compilador DML.
 - Utilidades: carga; *backup*; reorganización de ficheros; control de rendimiento.
- Transacciones, concurrencia, errores e integridad.
 - Gestor transaccional.
 - Propiedades ACID de las transacciones.
 - Control de concurrencia.
 - Problemas típicos de concurrencia: lectura sucia; modificación perdida; lectura no repetible; lectura fantasma.
 - Bloqueos: tipos y modos.
 - Niveles de aislamiento: lectura no confirmada; lectura confirmada; lectura repetible; secuenciable.
 - Recuperación de errores.
 - Copias de seguridad y bitácora.
 - Técnicas de recuperación: actualización diferida vs actualización inmediata.
 - Integridad.
- Principales SGBD: *Oracle*, *PostgreSQL* y *MySQL/MariaDB*. Conceptos: arquitectura; administración, gestión y optimización.
- Tendencias actuales:
 - Procesamiento híbrido transaccional-analítico (HTAP, *hybrid transactional/analytical processing*)
 - Bases de datos multimodelo.
 - Seguridad incorporada.
 - Bases de datos inteligentes y autónomas.

8 GLOSARIO.

- **4GL:** Estas siglas se emplean habitualmente para referirse a los lenguajes de cuarta generación.
- **ACID:** Siglas en inglés de *Atomicity, Consistency, Isolation, Durability* (atomicidad, consistencia, aislamiento y persistencia), que son las propiedades que deben cumplir las transacciones. No confundir las propiedades ACID de las transacciones con las propiedades ACID relativas a la seguridad de la información (Autenticidad, Confidencialidad, Integridad, Disponibilidad).
- **ANSI:** Siglas en inglés de *American National Standards Institute* (Instituto Nacional Estadounidense de Estándares).
- **ANSI/SPARC:** *American National Standards Institute / Standards Planning And Requirements Committee*. Es un estándar de diseño abstracto para un sistema de gestión de bases de datos.
- **DBA:** Siglas en inglés de *Data Base Administrator* (administrador de bases de datos).
- **DCL:** Siglas en inglés de *Data Control Language* (lenguaje de control de datos).
- **DDL:** Siglas en inglés de *Data Definition Language* (lenguaje de definición de datos).
- **DML:** Siglas en inglés de *Data Manipulation Language* (lenguaje de manipulación de datos).
- **DBMS:** Siglas en inglés de *Data Base Management System* (sistema de gestión de bases de datos).
- **SGBD:** Siglas de Sistema de Gestión de Bases de Datos.
- **SQL:** Siglas en inglés de *Structured Query Language* (lenguaje de consulta estructurado).

9 TEST.

9.1 PREGUNTAS DE TEST.

- 1) En la arquitectura ANSI/X3 SPARC a tres niveles, indicar cuál de los esquemas citados a continuación no corresponde a dicha arquitectura:
 - a) Esquema externo.
 - b) Esquema legal.
 - c) Esquema conceptual.
 - d) Esquema interno.
- 2) En relación con la definición y características de una base de datos, ¿cuál de las siguientes afirmaciones es falsa?
 - a) En una base de datos se admite la redundancia controlada.
 - b) En una base de datos no debe darse la redundancia lógica.
 - c) En una base de datos debe existir la redundancia física.
 - d) En una base de datos puede admitirse la redundancia lógica pero no física.
- 3) Entre los distintos niveles de abstracción de una base de datos ¿con cuál de los siguientes conceptos se identifica el "usuario final"?
 - a) Esquema externo.
 - b) Esquema conceptual.
 - c) Esquema interno.
 - d) Esquema global.
- 4) En una base de datos las vistas:
 - a) Definen la estructura y organización de los datos.
 - b) Permiten restringir el acceso, permitiendo que diferentes usuarios sólo vean ciertas filas o columnas de una tabla.
 - c) Se crean automáticamente cuando una consulta se realiza más de una vez en la misma sesión.
 - d) Sólo pueden ser creadas por el usuario propietario del esquema.
- 5) Indique la respuesta correcta:
 - a) El modelo jerárquico es más flexible que el de red o reticular.
 - b) El modelo relacional se basa en el álgebra y cálculo relacional.
 - c) Cronológicamente el modelo de red o reticular es anterior al jerárquico.
 - d) En el modelo relacional se exige que el usuario conozca las vinculaciones entre entidades.
- 6) El modelo de base de datos CODASYL:
 - a) Es el modelo conceptual de datos que corresponde a los SGBD jerárquicos.
 - b) Es el modelo lógico de datos de un SGBD relacional.
 - c) Es el modelo de Codd.
 - d) No se corresponde con nada de lo anterior.
- 7) En el modelo de referencia ANSI/SPARC, el nivel conceptual:
 - a) Se obtiene conceptualizando los datos físicos
 - b) Es una representación abstracta de la base de datos, común para todos los usuarios
 - c) Conceptualiza y abstrae las vistas del nivel externo.

PREGUNTAS DE TEST

- d) Es una representación abstracta de la base de datos, diferente para cada usuario.
- 8) Se puede decir que una base de datos es:
- a) Un elemento físico (hardware).
 - b) Un dispositivo de almacenamiento.
 - c) Un conjunto de datos interrelacionados.
 - d) Una colección de ficheros independientes entre sí.
- 9) En la arquitectura de tres niveles para el diseño de bases de datos establecida por el ANSI, ¿qué nivel se ocupa de las reglas de integridad, claves e índices?
- a) Nivel físico.
 - b) Nivel conceptual.
 - c) Nivel externo.
 - d) Ninguno de los tres niveles se ocupa de las reglas de integridad, claves e índices, lo hace el administrador de la base de datos.
- 10) ¿Cuál de las siguientes órdenes SQL pertenece al Lenguaje de Manipulación de Datos (DML) de un SGBD?
- a) CREATE TABLE.
 - b) REVOKE.
 - c) INSERT.
 - d) Todas las anteriores.
- 11) El Lenguaje de Definición de Datos describe:
- a) Las propiedades dinámicas de las entidades.
 - b) Las propiedades estáticas de las entidades.
 - c) Los dos tipos de propiedades, es indiferente.
 - d) No define propiedades, define comportamientos.
- 12) Si para consolidar una transacción en un sistema relacional se utiliza la sentencia COMMIT, ¿qué sentencia se utiliza para retroceder una transacción?
- a) NO COMMIT.
 - b) ROLLBACK.
 - c) END TRANSACTION.
 - d) BACK TRANSACTION.
- 13) ¿Cuál es la estructura más antigua de un modelo SGBD?
- a) Relacional.
 - b) En red.
 - c) Jerárquico.
 - d) Multidimensional.
- 14) CODASYL, modelo de Sistemas Gestores de Bases de datos en red, es acrónimo de:
- a) Conference on Data Systems Languages.
 - b) Concurrent Database Systems Languages.
 - c) Coherent Data Symmetric Location.
 - d) CODE And Systematic Language.

PREGUNTAS DE TEST

- 15) ¿Cuál de las siguientes herramientas utiliza un SGBD para asegurar la atomicidad y persistencia de las transacciones?
- a) Módulo de comprobación de la integridad.
 - b) El esquema conceptual.
 - c) Fichero diario o bitácora.
 - d) Triggers.
- 16) En una transacción en un Sistema de Gestión de Bases de Datos (SGBD), ¿qué se entiende por el punto de integridad?
- a) El momento después en el que se inicia la transacción.
 - b) El estado de la base de datos en el momento en que se inicia la transacción.
 - c) Hacer un rollback en el caso de que haya un problema antes de completar la transacción.
 - d) El retorno con la confirmación de que la transacción se ha completado con éxito.
- 17) En un SGBD (Sistema Gestor de Base de Datos), ¿qué tipo de bloqueos puede haber?
- a) Bloqueo de lectura, escritura y rango.
 - b) Bloqueo de lectura y escritura.
 - c) Bloqueo de lectura únicamente.
 - d) Bloqueo de rango y de conjunto.
- 18) El termino anglosajón "ACID" relacionado con una transacción de base de datos se corresponde con las propiedades:
- a) Authentication, Consistency, Integrity, Durability.
 - b) Atomicity, Confidentiality, Identification, Durability.
 - c) Atomicity, Consistency, Isolation, Durability.
 - d) Availability, Consistency, Isolation, Durability.
- 19) ¿Cuál de los siguientes niveles de aislamiento previene de los problemas de lecturas sucias y lecturas no repetibles, pero NO evita las lecturas fantasmas?
- a) Read Committed (lectura confirmada).
 - b) Serializable.
 - c) Read uncommitted (lectura no confirmada).
 - d) Repeatable read (lectura repetible).
- 20) En el entorno de las bases de datos relacionales se denomina ACID a:
- a) Un conjunto de características necesarias para que una serie de instrucciones puedan ser consideradas como una transacción.
 - b) El método que determina cuando una transacción finaliza con éxito.
 - c) El método que determina cuando una transacción no finaliza con éxito.
 - d) Ninguna de las anteriores.

9.2 SOLUCIONES A LAS PREGUNTAS DE TEST.

PREGUNTA	SOLUCIÓN
1	B
2	D
3	A
4	B
5	B
6	D
7	B
8	C
9	A
10	C
11	B
12	B
13	C
14	A
15	C
16	B
17	A
18	C
19	D
20	A

10 BIBLIOGRAFÍA BÁSICA.

10.1 BIBLIOGRAFÍA BÁSICA.

1. Date, C. J. (2001). Introducción a los sistemas de bases de datos. Addison-Wesley Iberoamericana.
2. Ramez, E. y Shamkant, N. (2007). Fundamentos de sistemas de bases de datos, quinta edición. Pearson Educación.

10.2 BIBLIOGRAFÍA PARA AMPLIAR EL TEMA.

1. Beynon-Davies, P. (2014). Sistemas de bases de datos. Reverte.
2. Codd, E. F. (1970). "A Relational Model of Data for Large Shared Data Banks". Disponible en <https://www.seas.upenn.edu/~zives/03f/cis550/codd.pdf>
3. Jardine, D. A. (1977). The ANSI/SPARC DBMS Model. North-Holland Pub. Co.
4. Thomas, C. y Carolyn, B. (2005). Sistemas de bases de datos: un enfoque práctico para diseño, implementación y gestión. Addison-Wesley Iberoamericana.